

CS 425 / ECE 428

Distributed Systems

Fall 2026

Aishwarya Ganesan w/ Ram Kesavan

Lecture 7: Peer-to-peer Systems I

Ack: Indranil Gupta

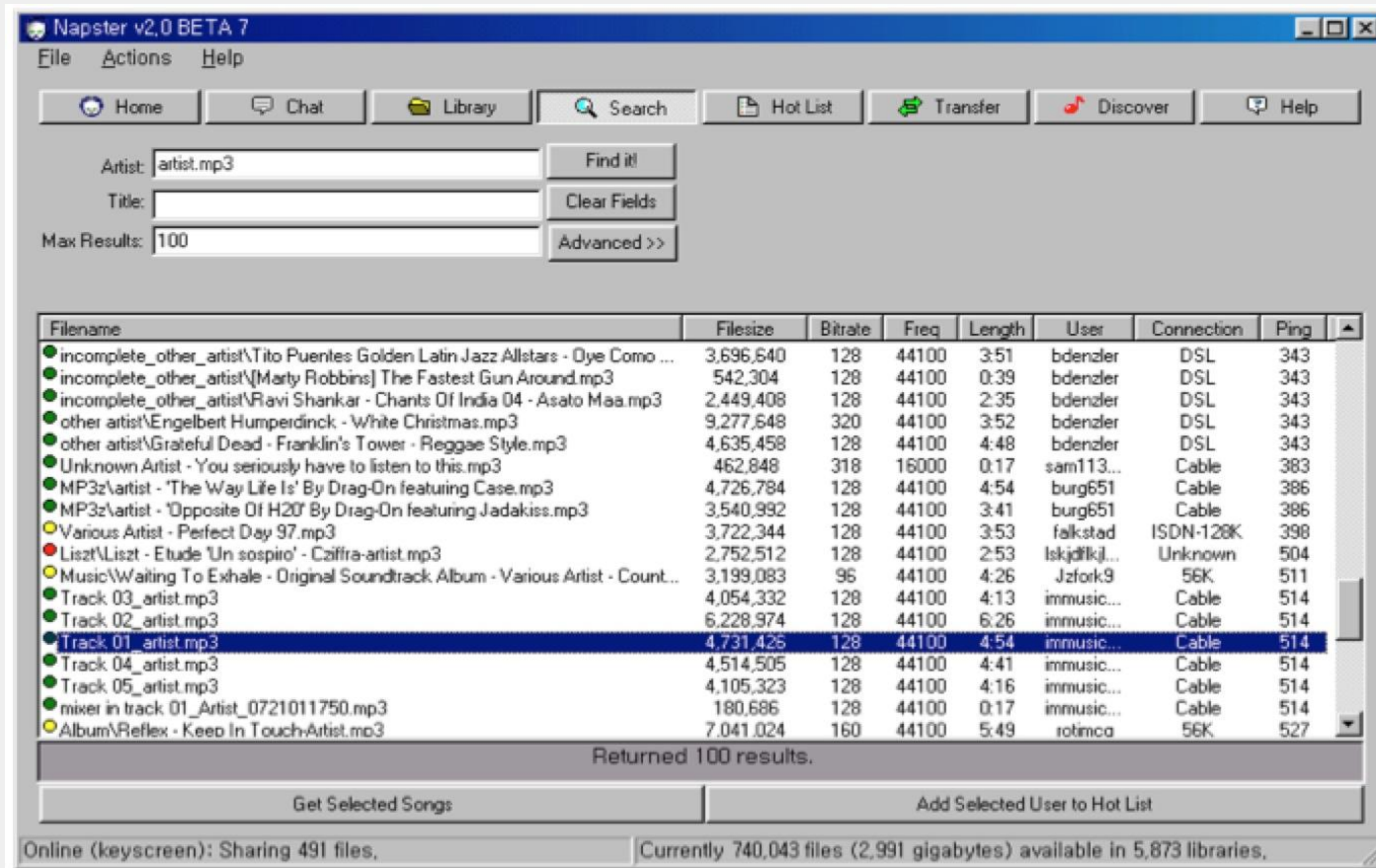
Announcements

- MP2 Released (due Sep 27)
- You will be implementing
 - Failure detector
 - Membership protocol
- Stage 2 (of 4) in building a fully-working distributed system from scratch
 - Stage 3 will be a distributed storage system
 - Stage 4 will be a full distributed system
- Pushed clarifications (in red) to HW1 (refresh the latest pdf)
- HW1 due Sun 9/20

Why Study Peer to Peer Systems?

- First distributed systems that seriously focused on scalability with respect to number of nodes
- P2P techniques abound in cloud computing systems
 - Key-value stores (e.g., Cassandra, Riak, Voldemort) use Chord p2p hashing

Napster UI



A Brief History

- [6/99] Shawn Fanning (freshman Northeastern U.) releases Napster online music service
- [12/99] RIAA sues Napster, asking \$100K per download
- [3/00] 25% UWisc traffic Napster, many universities ban it
- [00] 60M users
- [2/01] US Federal Appeals Court: users violating copyright laws, Napster is abetting this
- [9/01] Napster decides to run paid service, pay % to songwriters and music companies
- [Today] Napster protocol is open, people free to develop opennap clients and servers <http://opennap.sourceforge.net>
 - Gnutella: <http://www.limewire.com> (deprecated)
 - Peer to peer working groups: <http://p2p.internet2.edu>

What We Will Study

- Widely-deployed P2P Systems (This Lecture)
 1. Napster
 2. Gnutella
 3. BitTorrent
- P2P Systems with Provable Properties (Next Lecture)
 1. Chord
 2. Kelips

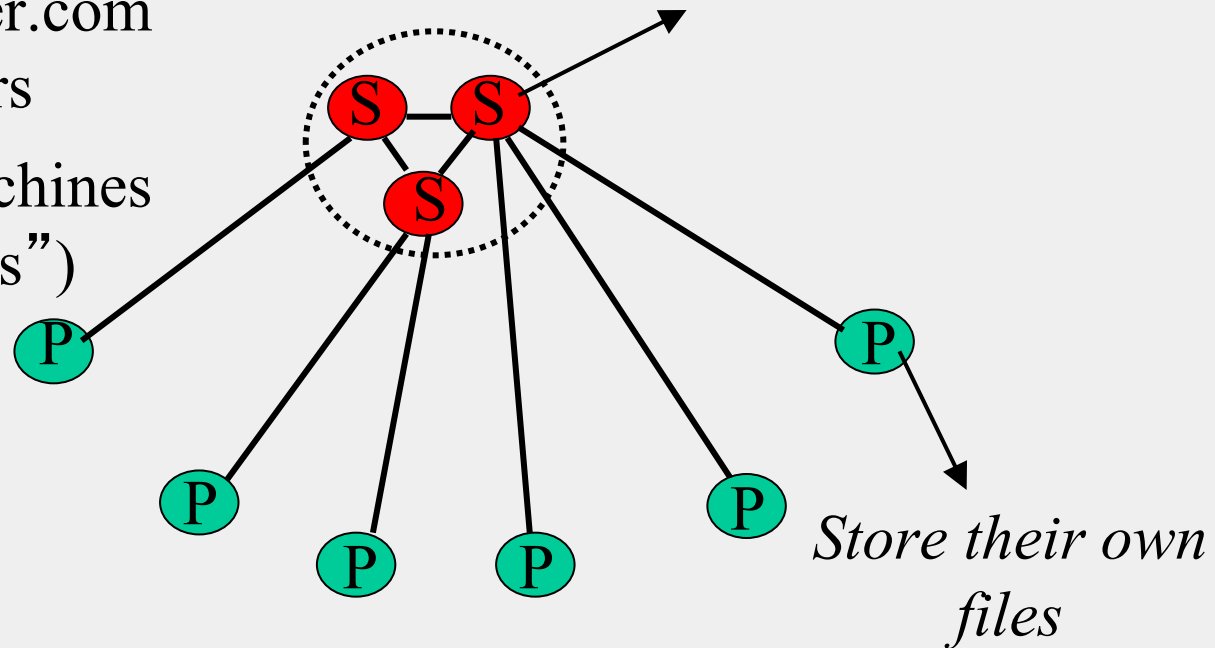
Napster Structure

*Store a directory, i.e.,
filenames with peer pointers*

Filename	Info about
PennyLane.mp3	Beatles, @ 128.84.92.23:1006

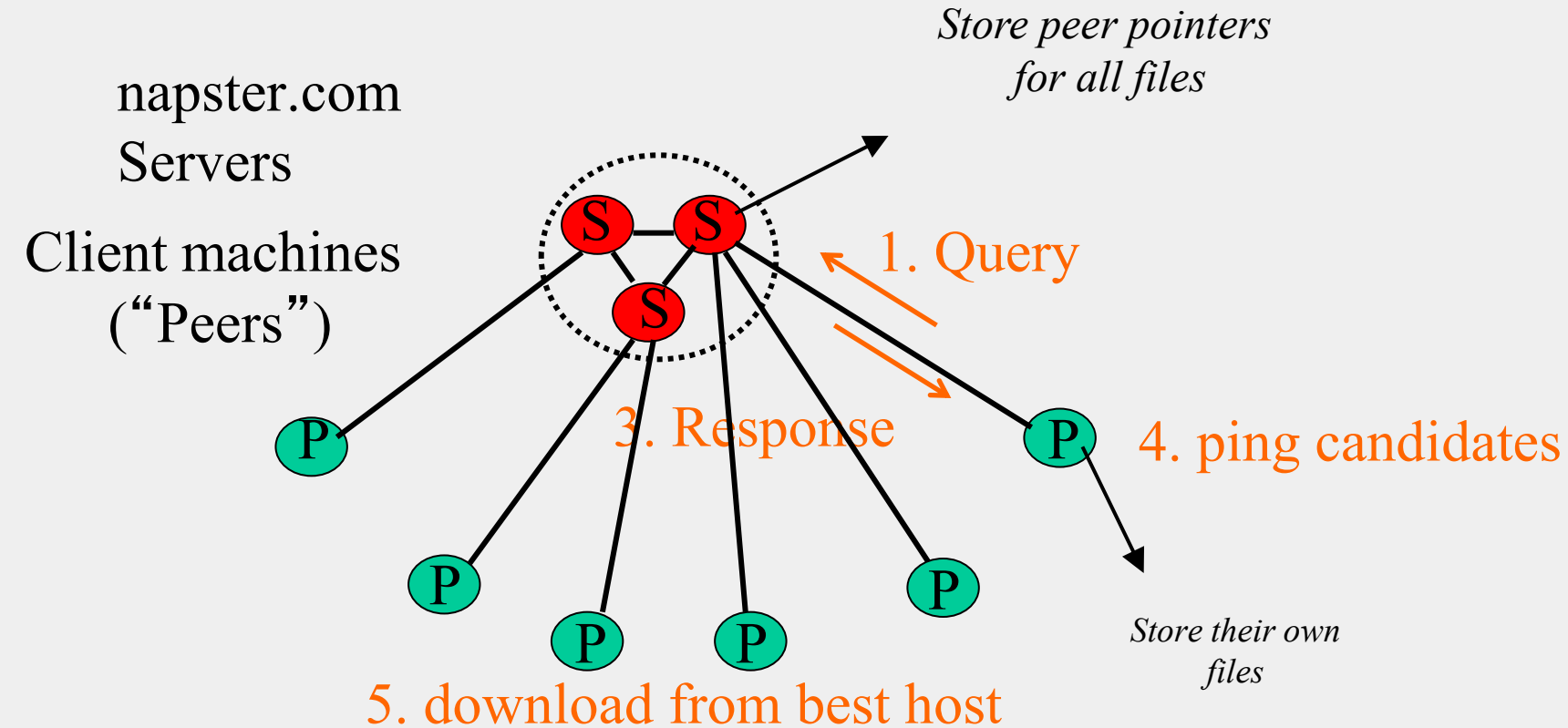
napster.com
Servers

Client machines
("Peers")



Napster Search

2. All servers search their lists (ternary tree algorithm)



Napster Operations

Client

- Connect to a Napster server
 - Upload list of music files that you want to share
 - Server maintains list of <filename, ip_address, portnum> tuples.
 - **Server stores no files.**

Napster Operations

Client (contd.)

- Search
 - Send server keywords to search with
 - (Server searches its list with the keywords)
 - Server returns a list of hosts - <ip_address, portnum> tuples - to client
 - Client pings each host in the list to find transfer rates
 - Client fetches file from best host
- All communication uses TCP (Transmission Control Protocol)
 - Reliable and ordered networking protocol

Joining a P2P system



Whose IP addresses will
napster.com domain map to?

- Can be used for any p2p system
 - Send an http request to well-known url for that P2P service - `http://www.myp2pservice.com`
 - Message routed (after lookup in DNS=Domain Name system) to **introducer**
 - a well-known server that keeps track of some recently joined nodes in p2p system
 - Introducer initializes new peers' neighbor table

Problems



Why did centralized server design work well in YARN's RM or HDFS' NameNode?

- Centralized server a source of congestion
- Centralized server single point of failure
- No security: plaintext messages and passwds
- napster.com declared to be responsible for users' copyright violation
 - “Indirect infringement”
 - Next system: Gnutella

CS 425 / ECE 428

Distributed Systems

Fall 2026

Aishwarya Ganesan w/ Ram Kesavan

Lecture 7-8: Peer-to-peer Systems

Ack: Indranil Gupta

Peer to Peer Systems

- First distributed systems to seriously focus on scalability with respect to number of nodes
 - Massive scale
 - Anyone with a computer can run a process that joins the p2p system
 - Churn high – nodes joining and leaving
- P2P techniques abound in cloud computing systems

What We Will Study

- Widely-deployed P2P Systems
 1. Napster
 2. Gnutella
 3. BitTorrent
- P2P Systems with Provable Properties
 1. Chord
 2. Kelips

<https://pe.app/aishwaryaganesa>





In Napster, if one file suddenly is extremely popular, which resource is necessarily the bottleneck?

Responses hidden



Anonymous 119

The central server's storage, because it stores a copy of every file

The peer(s) possessing the popular file, because the server only performs lookup

The central server's outgoing bandwidth, because it sends the file

The central server's CPU, because every byte passes through it





Respond at

pe.app/aishwaryaganesa

 Poll Everywhere

What happens if the introducer node fails?

 Responses hidden



Anonymous 134

All existing peers are disconnected from the network

Existing peers can continue searching, sharing, and adding files

Existing peers can search but can no longer share files

New peers may be unable to join through that introducer



 Exit

< 4/10 >

☒ Choices

☐ Results

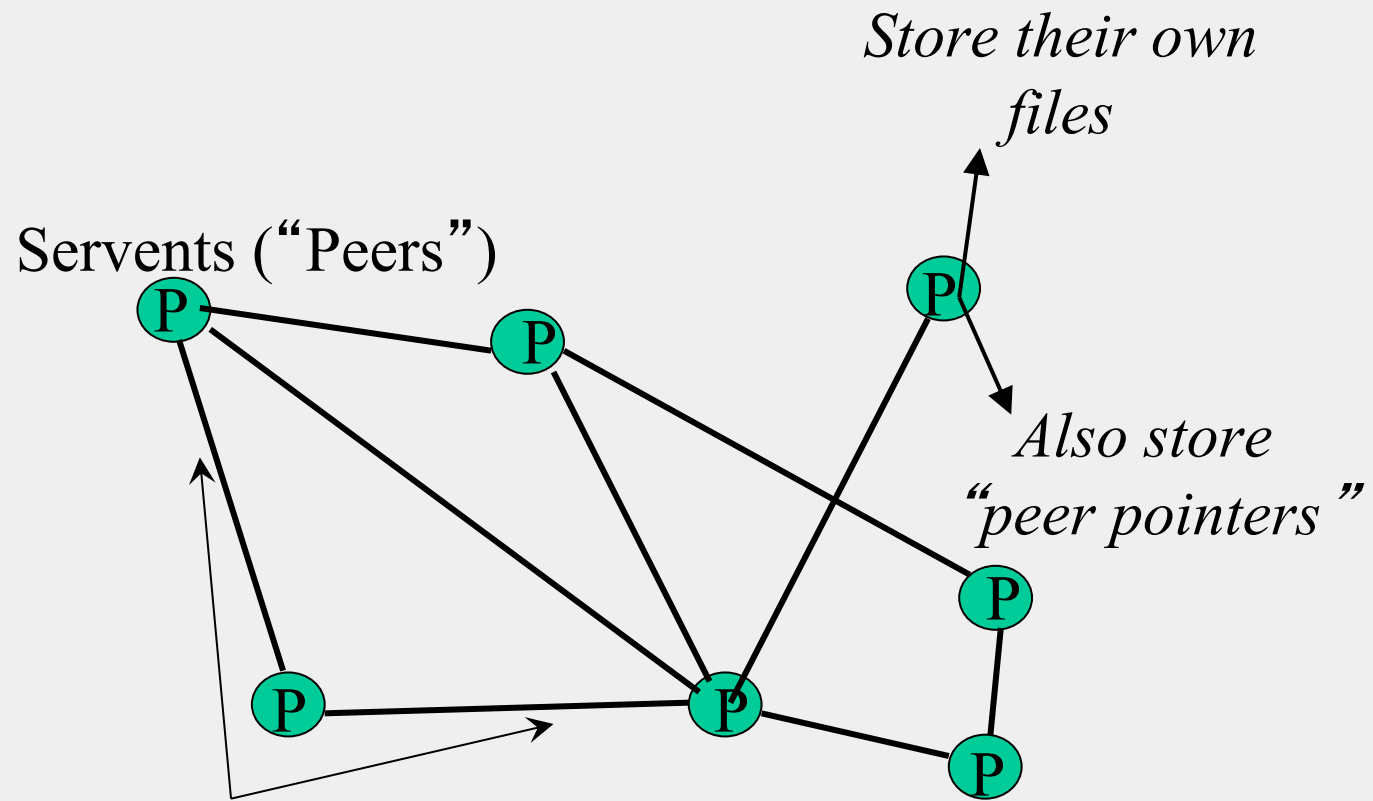
☐ Lock

☐ Correctness

Gnutella

- Eliminate the servers
- Client machines search and retrieve amongst themselves
- Clients act as servers too, called **servents**
- [3/00] release by AOL, immediately withdrawn, but 88K users by 3/03
- Original design underwent several modifications

Gnutella

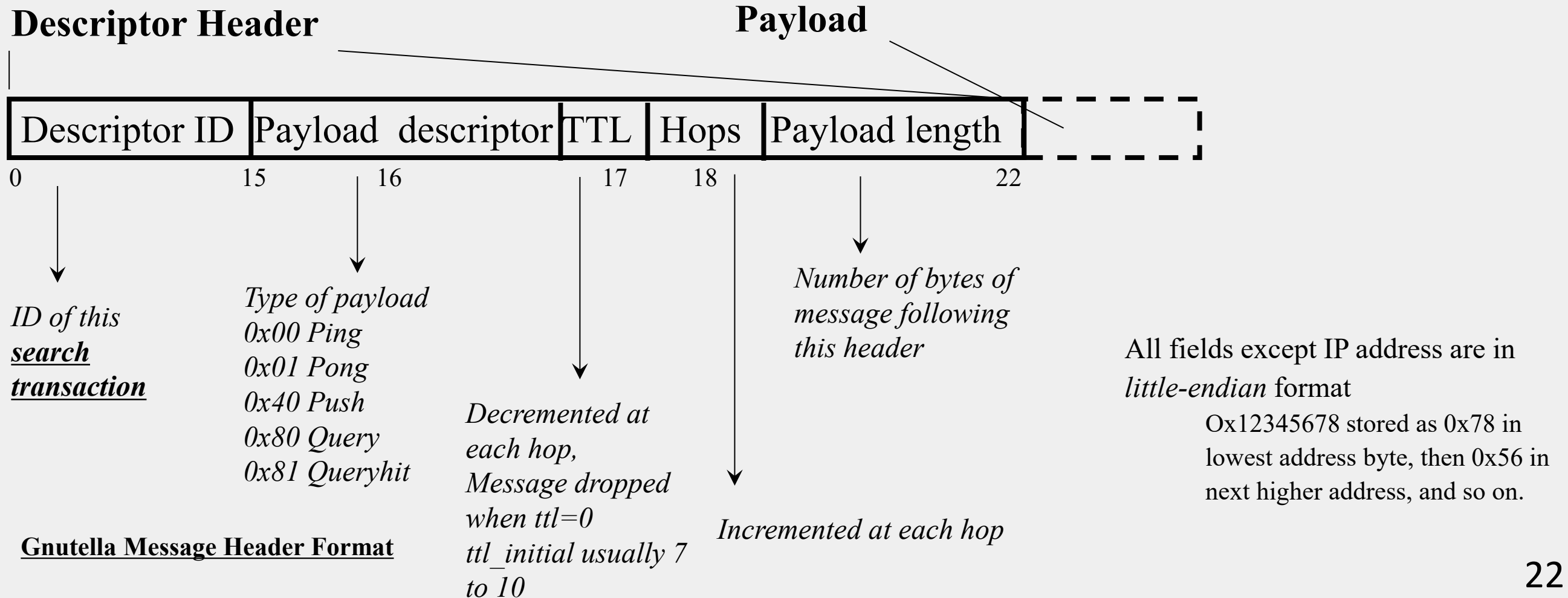


Connected in an **overlay graph**
(== each link is an implicit Internet path)

How do I search for my Beatles file?

- Gnutella *routes* different messages within the overlay graph
- Gnutella protocol has 5 main message types
 - **Query** (search)
 - **QueryHit** (response to query)
 - **Ping** (to probe network for other peers)
 - **Pong** (reply to ping, contains address of another peer)
 - Push (used to initiate file transfer)

Message Structure



How do I search for my Beatles file?

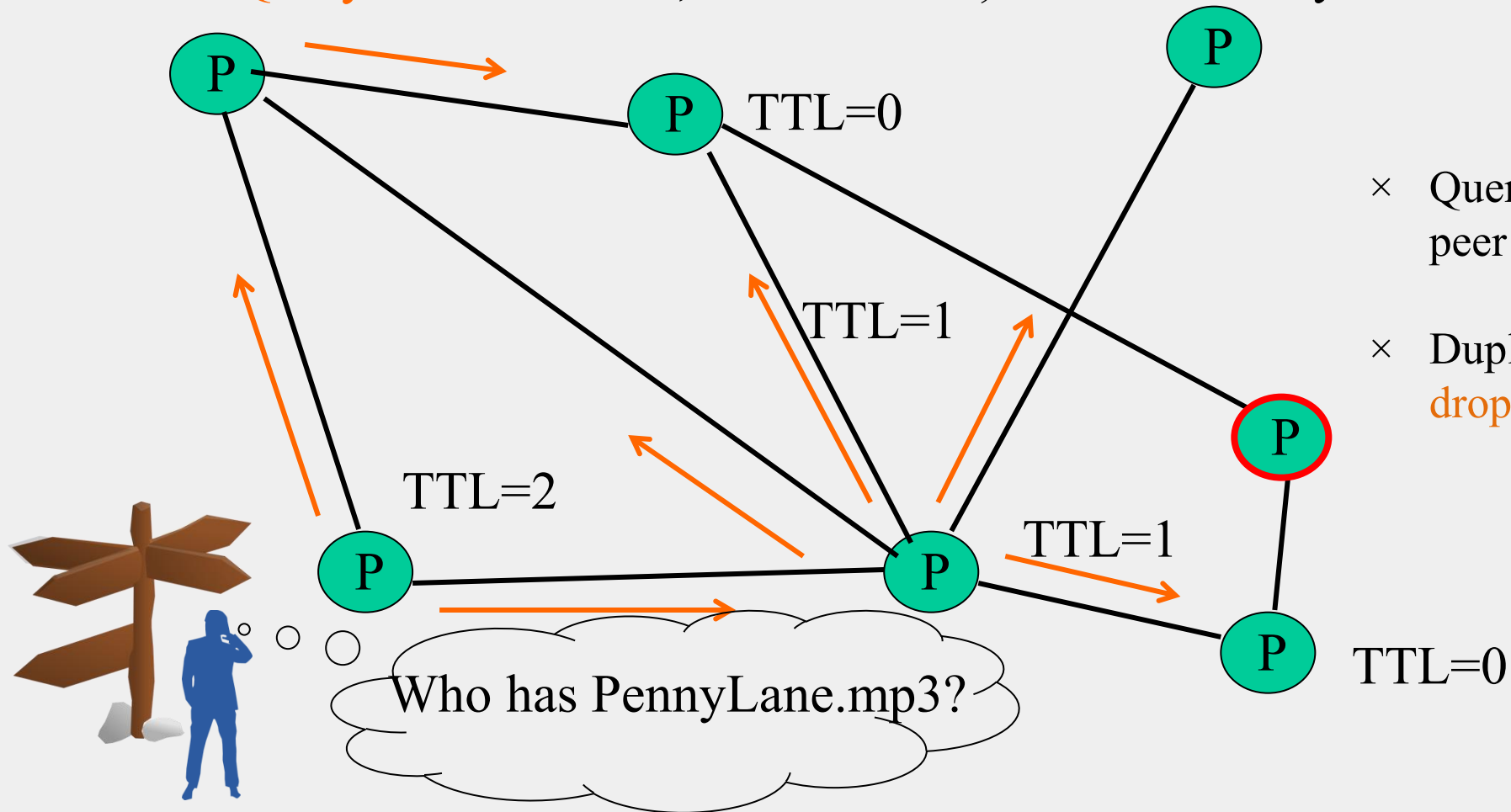
Query (0x80)



Payload Format in Gnutella **Query** Message

Gnutella Search

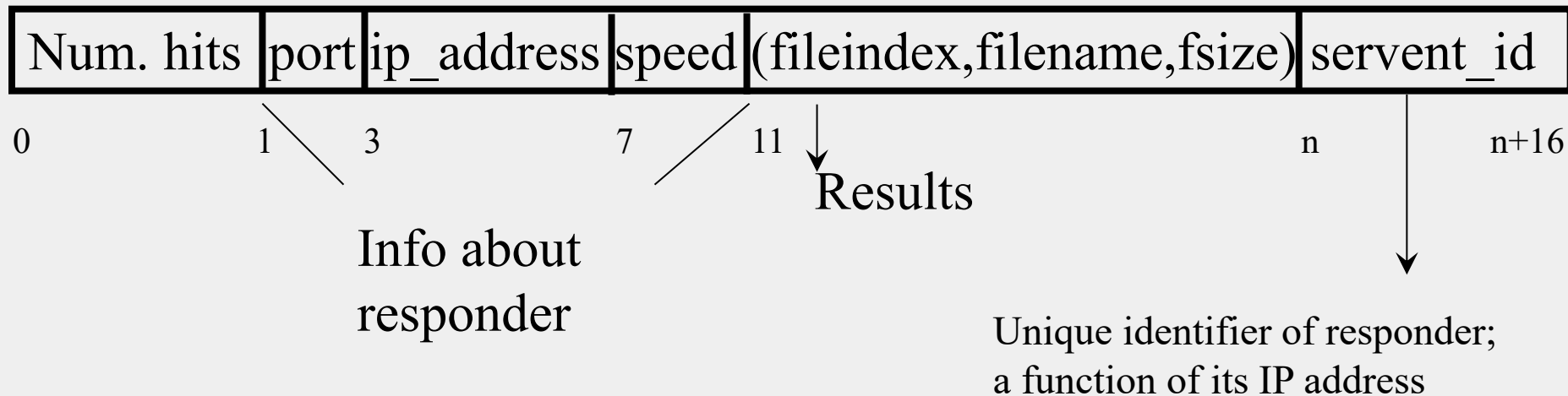
Query's flooded out, ttl-restricted, forwarded only once



- × Query forwarded to all neighbors **except** peer from which received
- × Duplicates with same DescriptorID are **dropped**

Gnutella Search

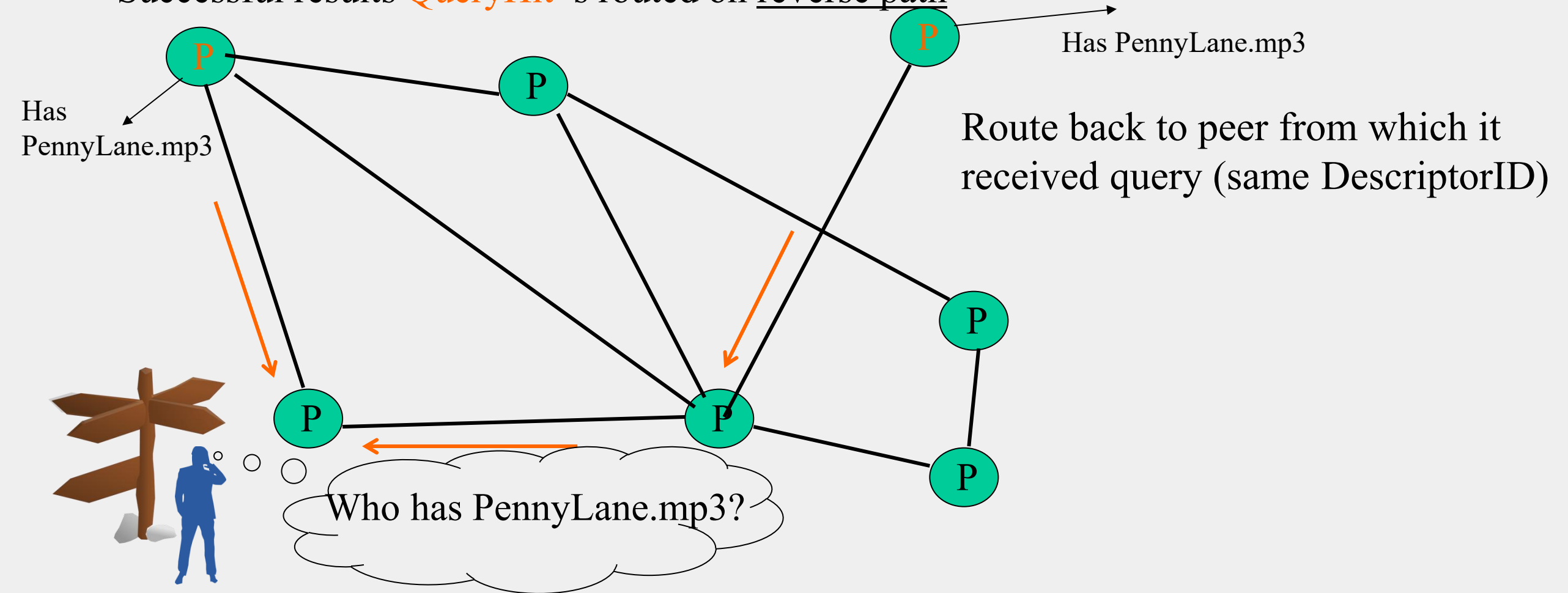
QueryHit (0x81) : successful result to a query



Payload Format in Gnutella **QueryHit** Message

Gnutella Search

Successful results **QueryHit**'s routed on reverse path



After receiving QueryHit messages

- Requestor chooses “best” QueryHit responder

- Initiates HTTP request **directly** to responder’s ip+port
 - (transfers not routed through Gnutella network)

GET /get/<File Index>/<File Name>/HTTP/1.0\r\n

Connection: Keep-Alive\r\n

Range: bytes=0-\r\n

User-Agent: Gnutella\r\n

\r\n

- Responder then replies with file packets after this message:

HTTP 200 OK\r\n

Server: Gnutella\r\n

Content-type:application/binary\r\n

Content-length: 1024 \r\n

\r\n

After receiving QueryHit messages (2)

- HTTP is the file transfer protocol. Why?
 - Because it's standard, well-debugged, and widely used.
- The “range” field in the GET request supports partial file transfers.
- What if responder is behind firewall that disallows incoming connections?



In Gnutella, if we increase TTL, which statement is most accurate?

Responses hidden



Anonymous 133

Search success probability decreases, but message overhead decreases

Search success probability increases, but message overhead can grow rapidly

Search success probability increases, while message overhead remains approximately constant

Search success probability and message overhead both decrease





Gnutella peer receives the same Query (same DescriptorID) from two different neighbors. What should it do when it receives the second copy?

Responses hidden



Anonymous 126

Forward it to all neighbors except the second sender

Forward it only if its TTL > 0

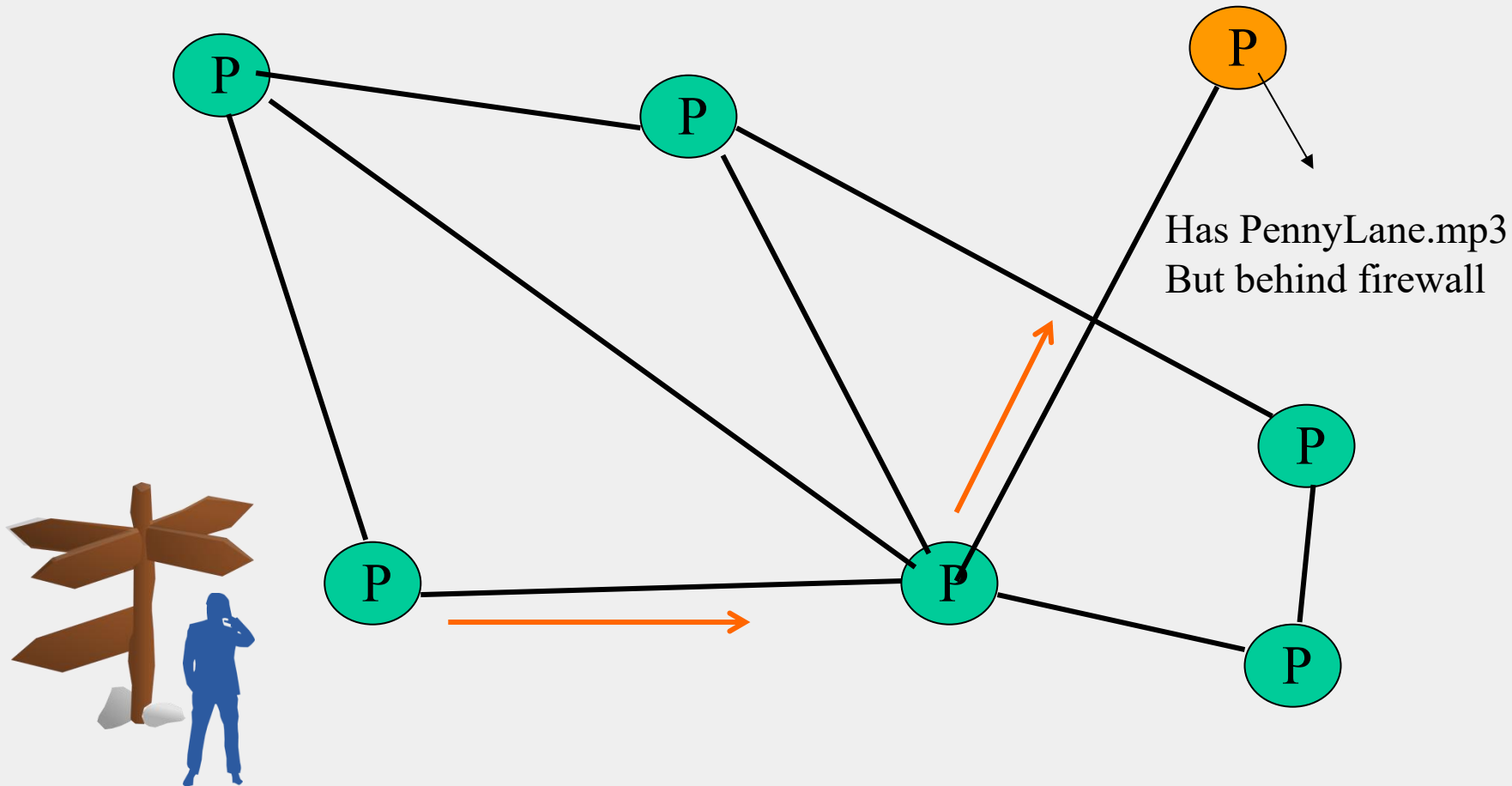
Drop it

Send a QueryHit back saying the query is duplicated



Dealing with Firewalls

Requestor sends **Push** to responder asking for file transfer



Dealing with Firewalls

Push (0x40)



same as in
received QueryHit

Address at which
requestor can accept
incoming connections

Dealing with Firewalls

- Responder establishes a TCP connection at ip_address, port specified. Sends

GIV <File Index>:<Servent Identifier>/<File Name>\n\n

- Requestor then sends GET to responder (as before) and file is transferred as explained earlier
- What if requestor is behind firewall too?

Ping-Pong

Ping (0x00)

no payload

Pong (0x01)

Port	ip_address	Num. files shared	Num. KB shared
------	------------	-------------------	----------------

Goal: keep neighbor lists fresh despite **peers joining, leaving and failing**

Peers initiate Ping's periodically

Pings flooded out like Querys

Pongs routed along reverse path like QueryHits

Pong replies used to **update set of neighboring peers**

List size specified by user at peer

- Heterogeneity: some peers may have more neighbors
- Gnutella found to follow **power law** distribution

Design choices: Avoiding excessive traffic



On Queryhit, what if peer that sent the query is no longer your neighbor?

To avoid duplicate transmissions, each peer maintains a list of **recently received messages**

- Query:
 - forwarded to all neighbors **except** peer from which received
 - same DescriptorID forwarded **only once**
- QueryHit:
 - routed back **only** to peer from which Query received with same DescriptorID
 - QueryHit with DescriptorID for which Query **not seen** is dropped

Duplicates with same DescriptorID, Message type are dropped

Gnutella Summary

- No servers
- Peers/servents maintain “neighbors”, this forms an overlay graph
- Peers store their own files
- Search through flooding queries to neighbors
- Periodic Ping-pong to continuously refresh neighbor lists

Gnutella Problems

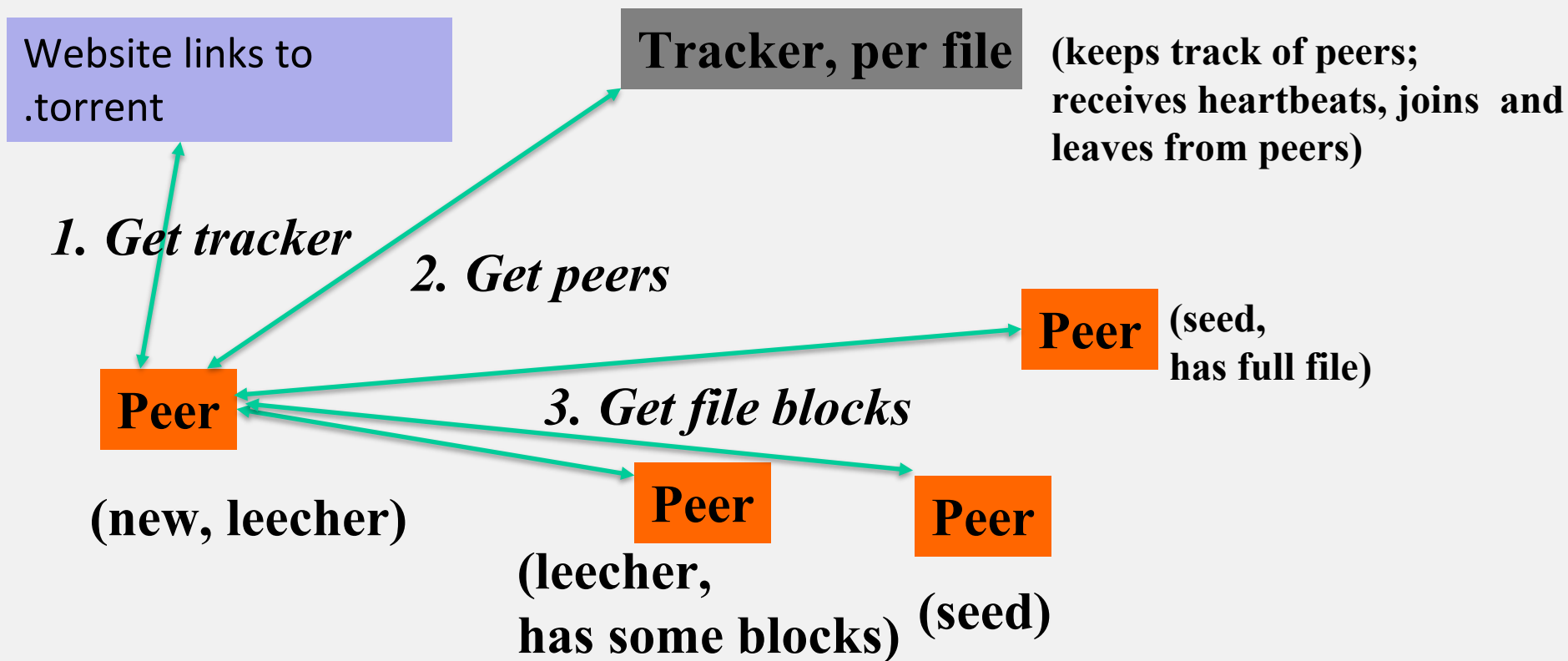
- Ping/Pong constituted 50% traffic
 - Solution: Multiplex, *cache* and reduce frequency of pings/pongs
- Repeated searches with same keywords
 - Solution: *Cache* Query, QueryHit messages
- Modem-connected hosts do not have enough bandwidth
 - Solution: use a central server to act as proxy for such peers
 - Another solution: FastTrack System (see Additional slides)

Gnutella Problems (contd.)

- Large number of *freeloaders*
 - 70% of users in 2000 were freeloaders
 - Only download files, never upload own files
- Flooding causes excessive traffic
 - Is there some way of maintaining meta-information about peers that leads to more intelligent routing?
 - ➔ Structured Peer-to-peer systems (e.g., Chord)

BitTorrent

? Advantages of splitting files into blocks?



BitTorrent: Policies

File split into blocks (32 KB – 256 KB)

Download **Local Rarest First** block policy:

- Prefer early download of blocks that are *least replicated* among neighbors

- Exception: New node allowed to pick one random neighbor: helps in bootstrapping

- Local and not global – decentralized

Tit for tat bandwidth usage:

- Provide blocks to neighbors that provided it the *best download* rates

- Incentive** for nodes to provide good upload rates

- Discourages freeloading

Choking: *Limit* number of neighbors to which *concurrent* uploads (e.g., 5) i.e., the “*best*” neighbors

- Everyone else choked

- Periodically re-evaluate this set (e.g., every 10 s) to see if the best peers have changed

Optimistic unchoking

- periodically (e.g., ~30 s), *unchoke a random choked* neighbor (need not be best)

- helps discover better peers

Announcements

- MP1 reports being graded
- MP2 out already, due 9/27 (demos on 9/28)
- HW1 due Sun 9/20
- HW2 will be out then

What We Are Studying

- Widely-deployed P2P Systems
 1. Napster
 2. Gnutella
 3. BitTorrent
- P2P Systems with Provable Properties
 1. Chord
 2. Kelips

Comparative Performance

 When is lookup latency $O(N)$ in Gnutella?

	Memory	Lookup Latency	#Messages for a lookup	
Napster	$O(1)$ ($O(N)$ @server)	$O(1)$	$O(1)$	
Gnutella	$O(N)$	$O(N)$	$O(N)$	

Comparative Performance

	Memory	Lookup Latency	#Messages for a lookup	
Napster	$O(1)$ $(O(N)@server)$	$O(1)$	$O(1)$	
Gnutella	$O(N)$	$O(N)$	$O(N)$	
Chord	$O(\log(N))$	$O(\log(N))$	$O(\log(N))$	

Chord

- Developers: I. Stoica, D. Karger, F. Kaashoek, H. Balakrishnan, R. Morris, Berkeley and MIT
- Intelligent choice of neighbors to reduce latency and message cost of routing (lookups/inserts)

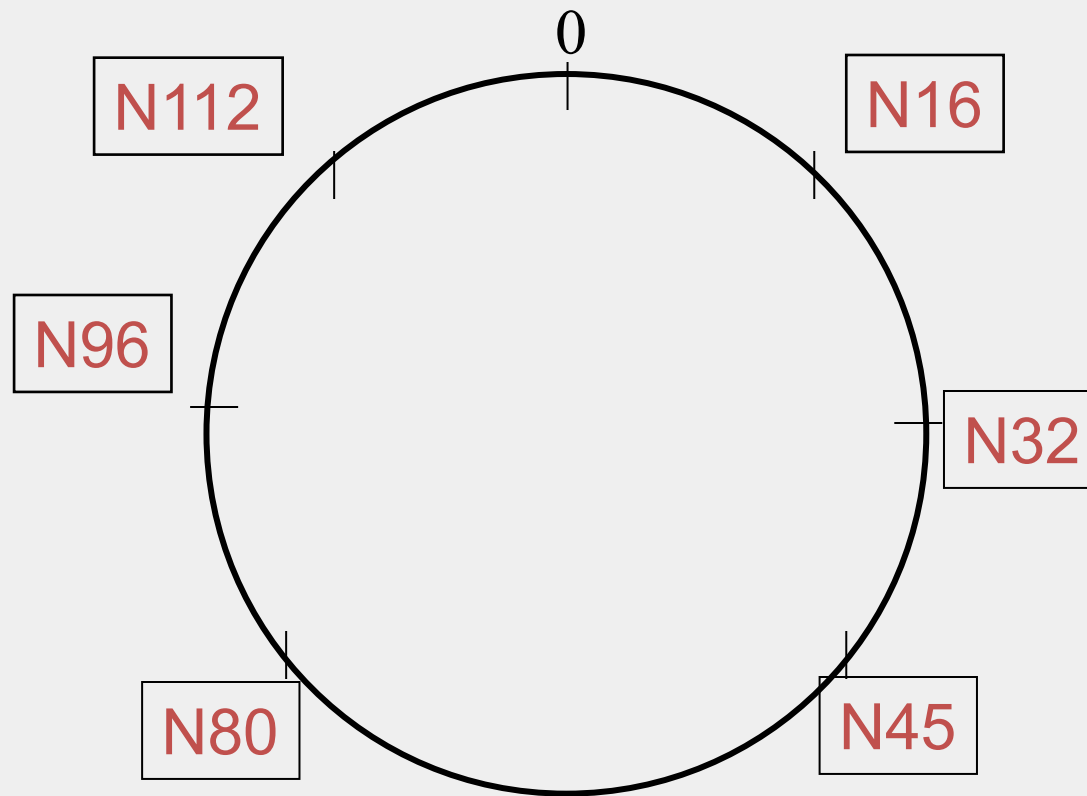
DHT=Distributed Hash Table

- A hash table allows you to insert, lookup and delete objects with keys
- A *distributed* hash table allows you to do the same in a distributed setting (objects=files)
- Performance Concerns:
 - Load balancing
 - Fault-tolerance
 - Efficiency of lookups and inserts
 - Locality
- Napster, Gnutella are all DHTs (sort of)

Chord Uses Consistent Hashing

Say $m=7$

6 nodes



Uses *Consistent Hashing* on node's address

$\text{SHA-1}(\text{ip_address, port}) \rightarrow 160 \text{ bit string}$

Truncated to m bits

Called *peer id* (number between 0 and $2^m - 1$)

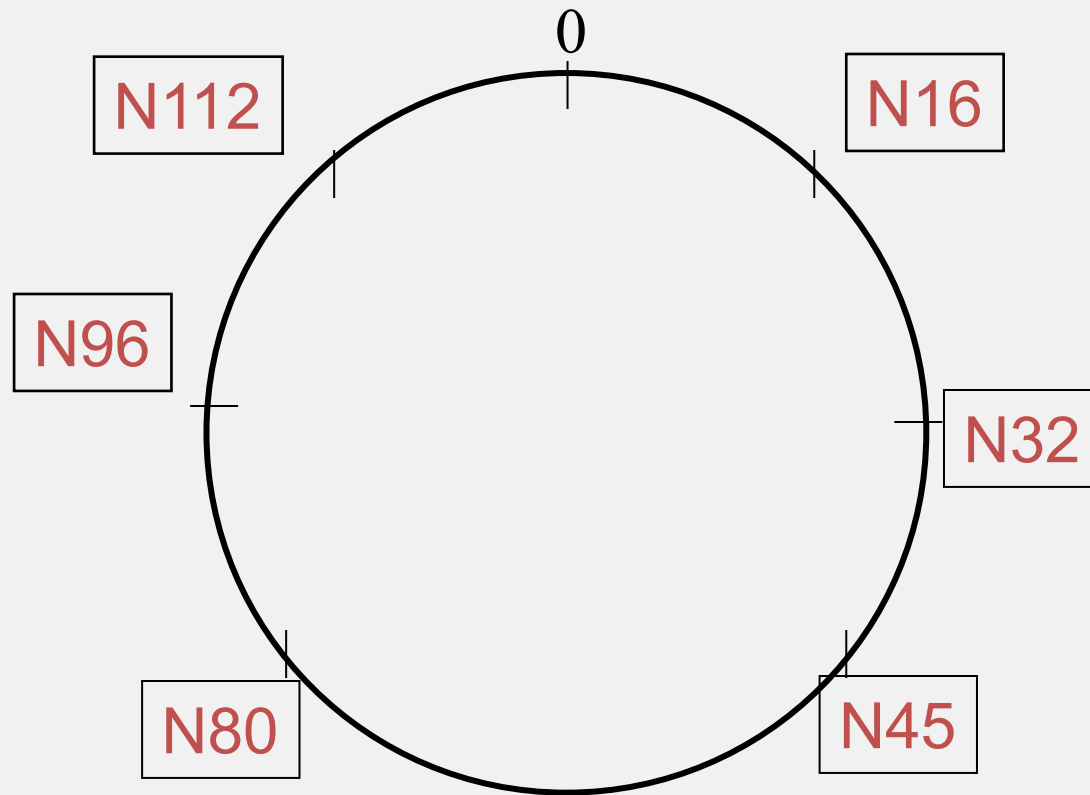
Not unique but id conflicts very unlikely

Can then map peers to one of 2^m logical points on a circle

Peer pointers (1): successors

Say $m=7$

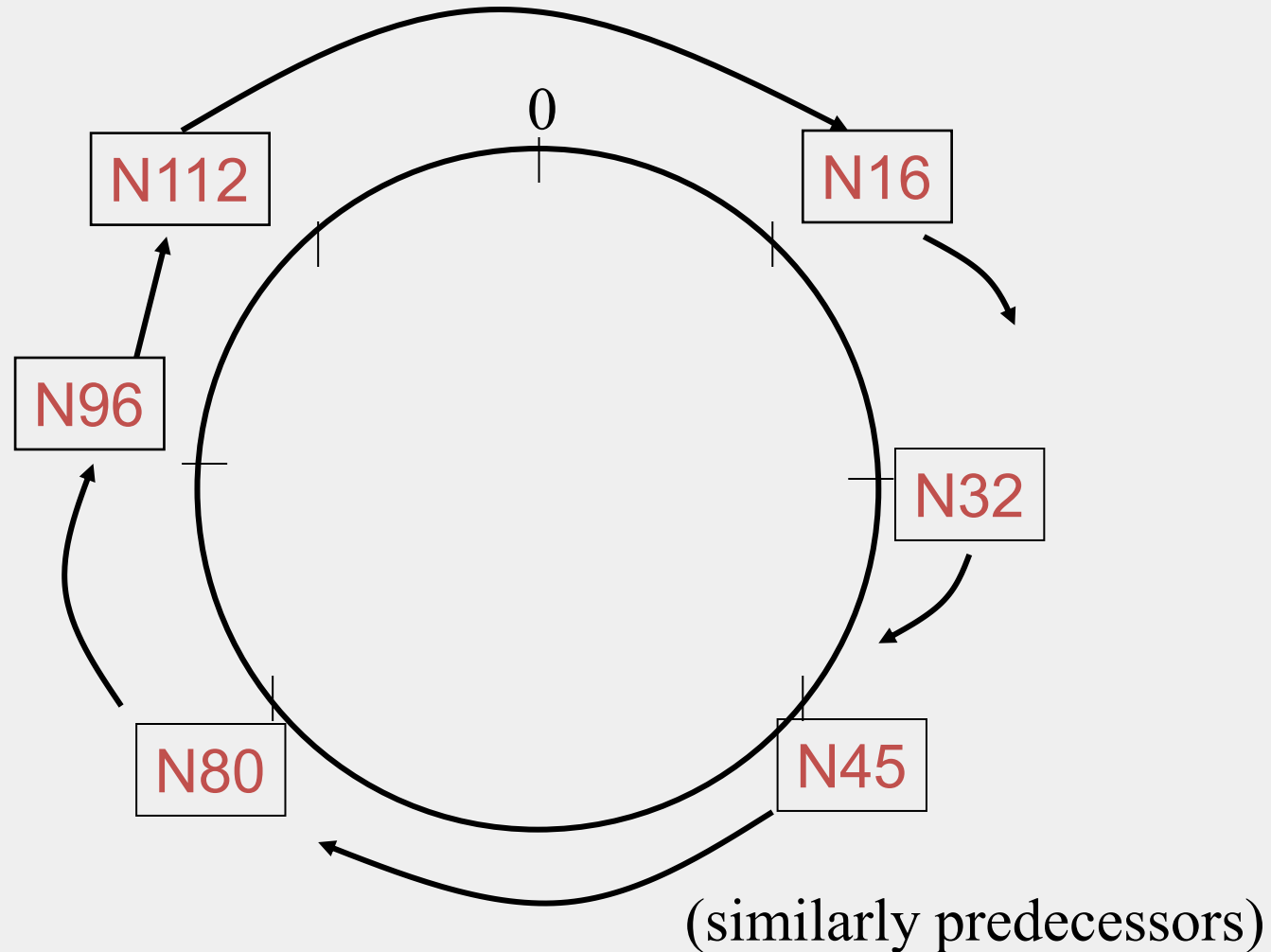
6 nodes



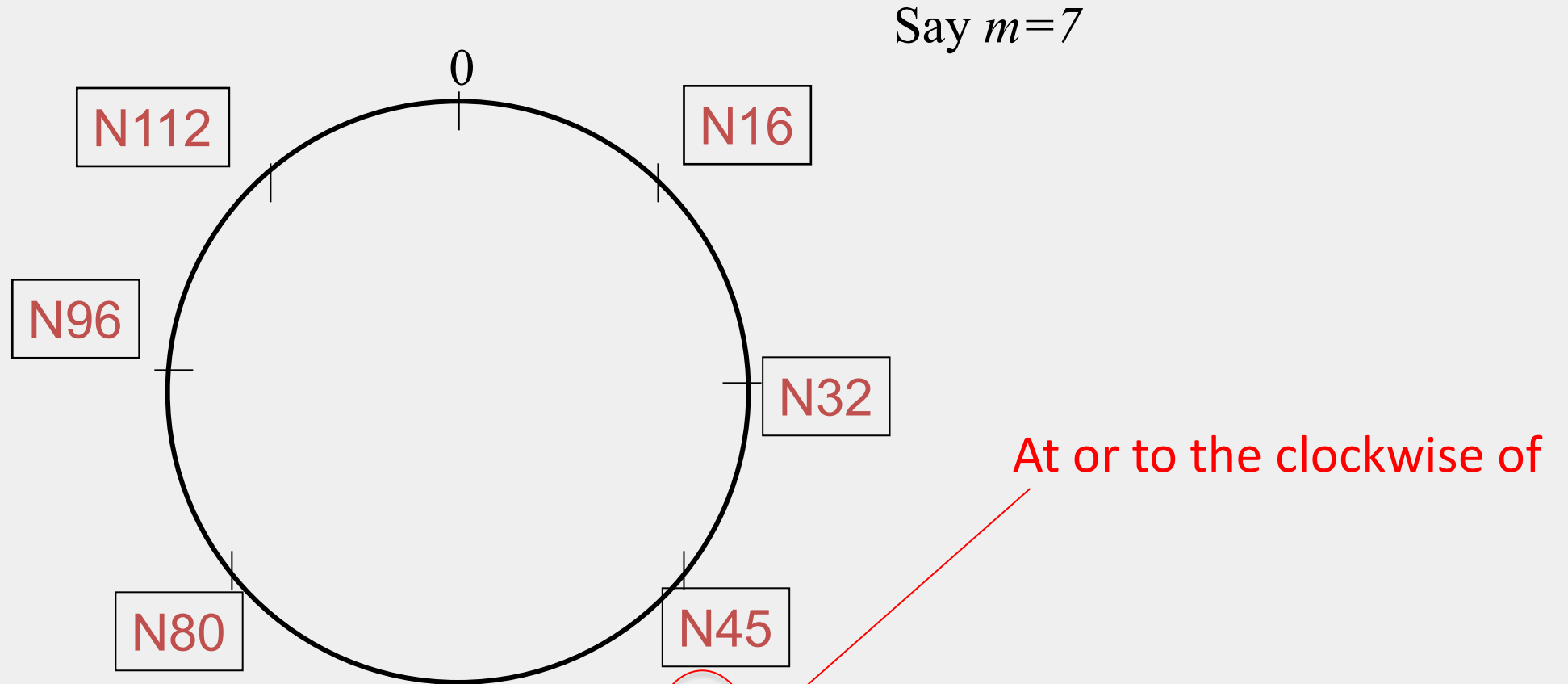
Peer pointers (1): successors

? Each peer forwards a search query to its successor. Does this work?

Say $m=7$



Peer pointers (2): finger tables



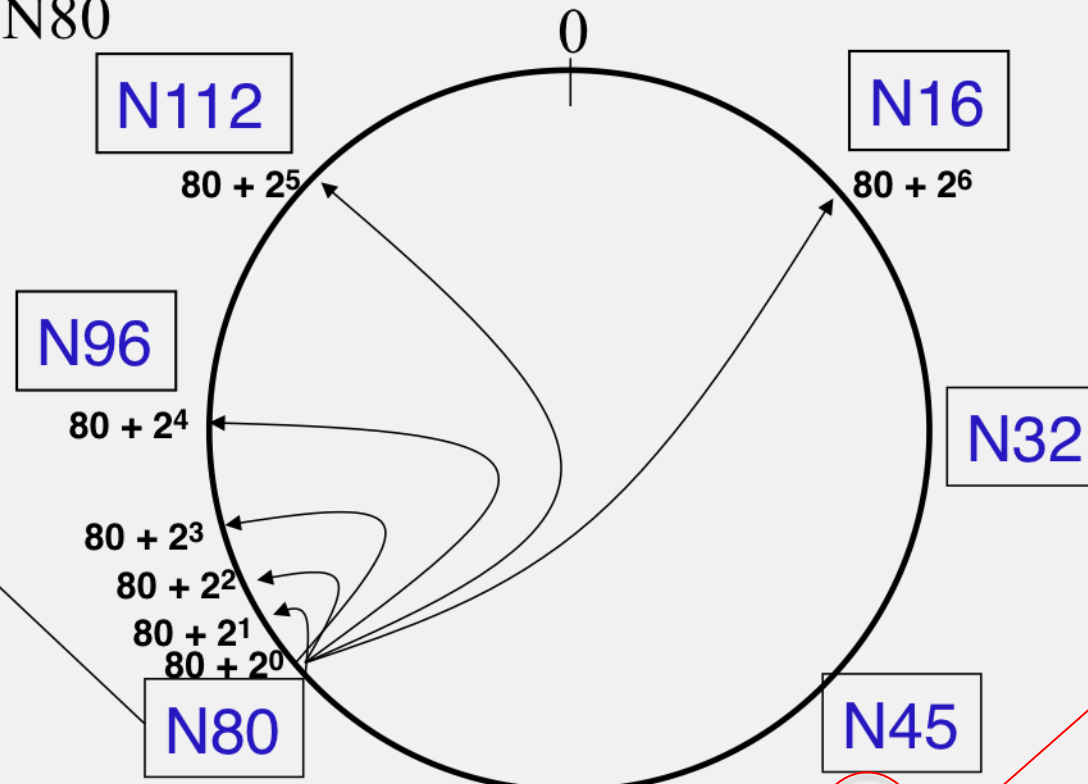
i^{th} entry at peer with id n is the first peer with id $\geq (n+2^i) \bmod 2^m$

Peer pointers (2): finger tables

Say $m=7$

Finger Table at N80

i	$ft[i]$
0	96
1	96
2	96
3	96
4	96
5	112
6	16



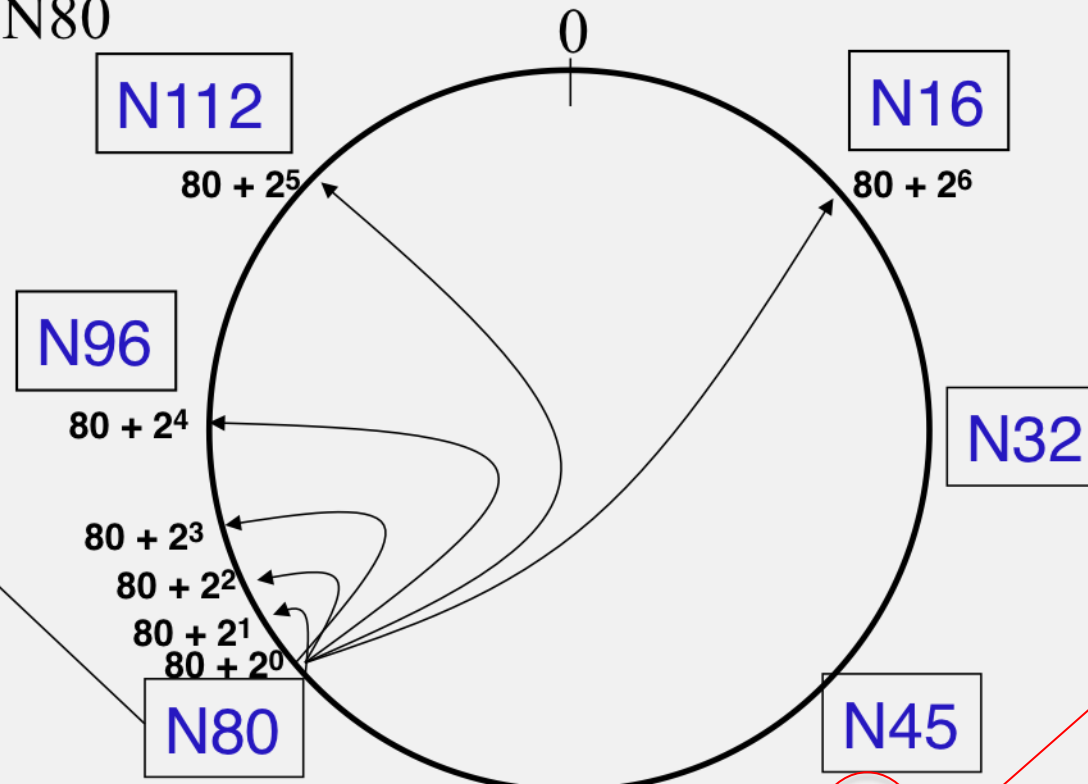
i^{th} entry at peer with id n is the first peer with id $\geq (n+2^i) \bmod 2^m$

Peer pointers (2): finger tables

? If node N112 doesn't exist, what's N80's 5th finger table entry?

Finger Table at N80

i	$ft[i]$
0	96
1	96
2	96
3	96
4	96
5	112
6	16



Say $m=7$

At or to the clockwise of

i^{th} entry at peer with id n is the first peer with id $\geq (n+2^i) \bmod 2^m$

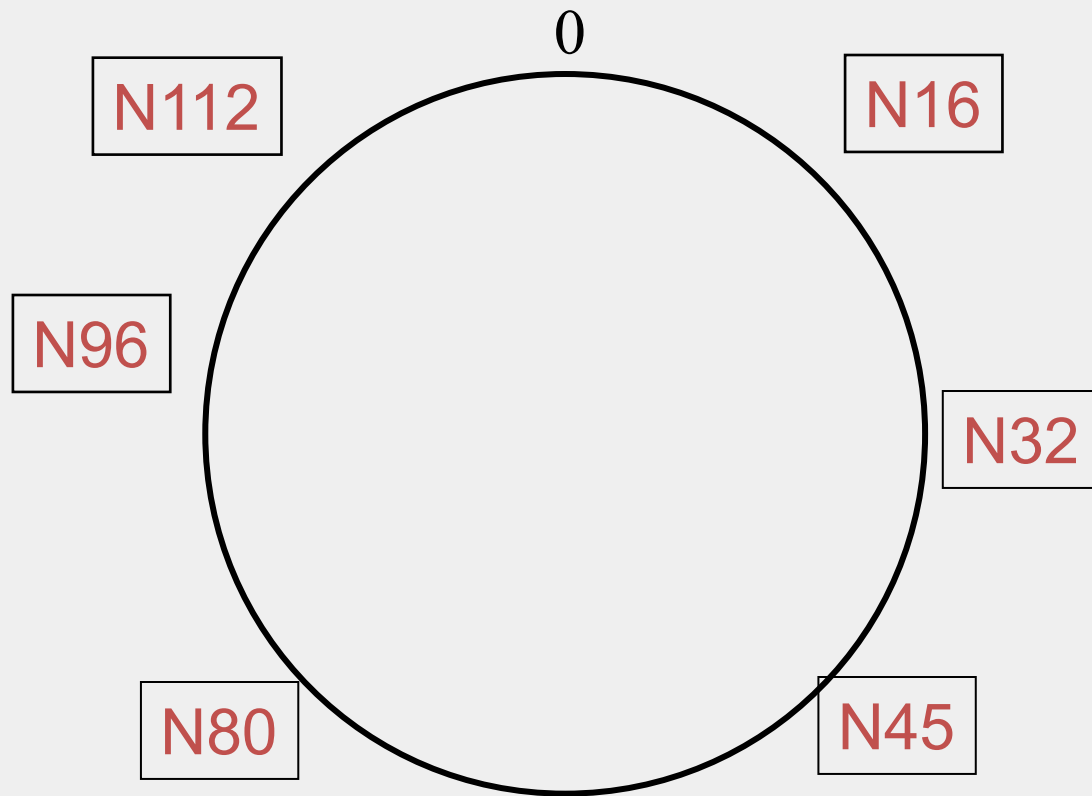
What about the files?

- Filenames also mapped using same consistent hash function
 - $\text{SHA-1}(\text{filename}) \rightarrow 160 \text{ bit string } (key)$
 - File is stored at first peer with id greater than or equal to its $key \bmod 2^m$



Mapping Files

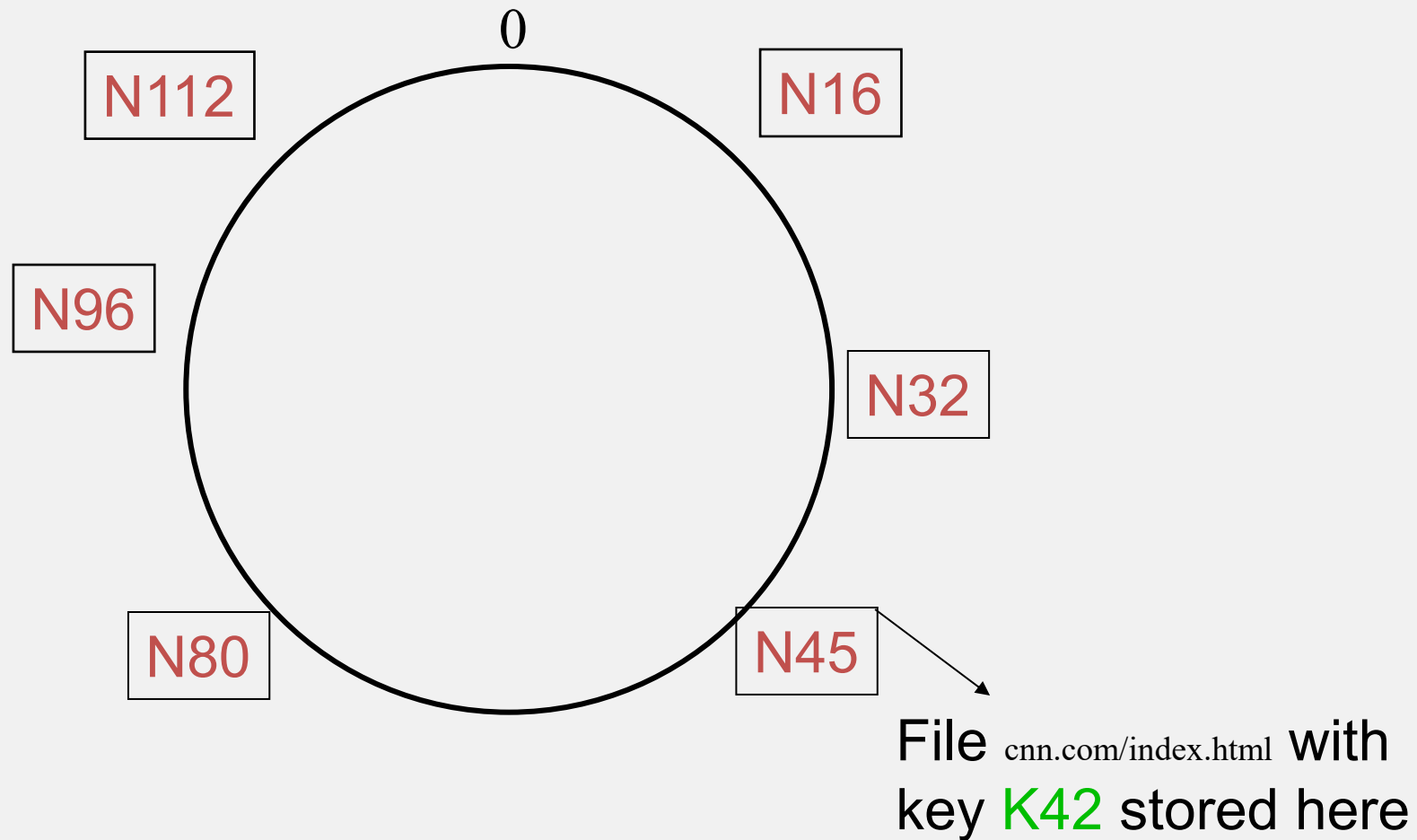
Say $m=7$



File *cnn.com/index.html* that maps to key K42

Mapping Files

Say $m=7$



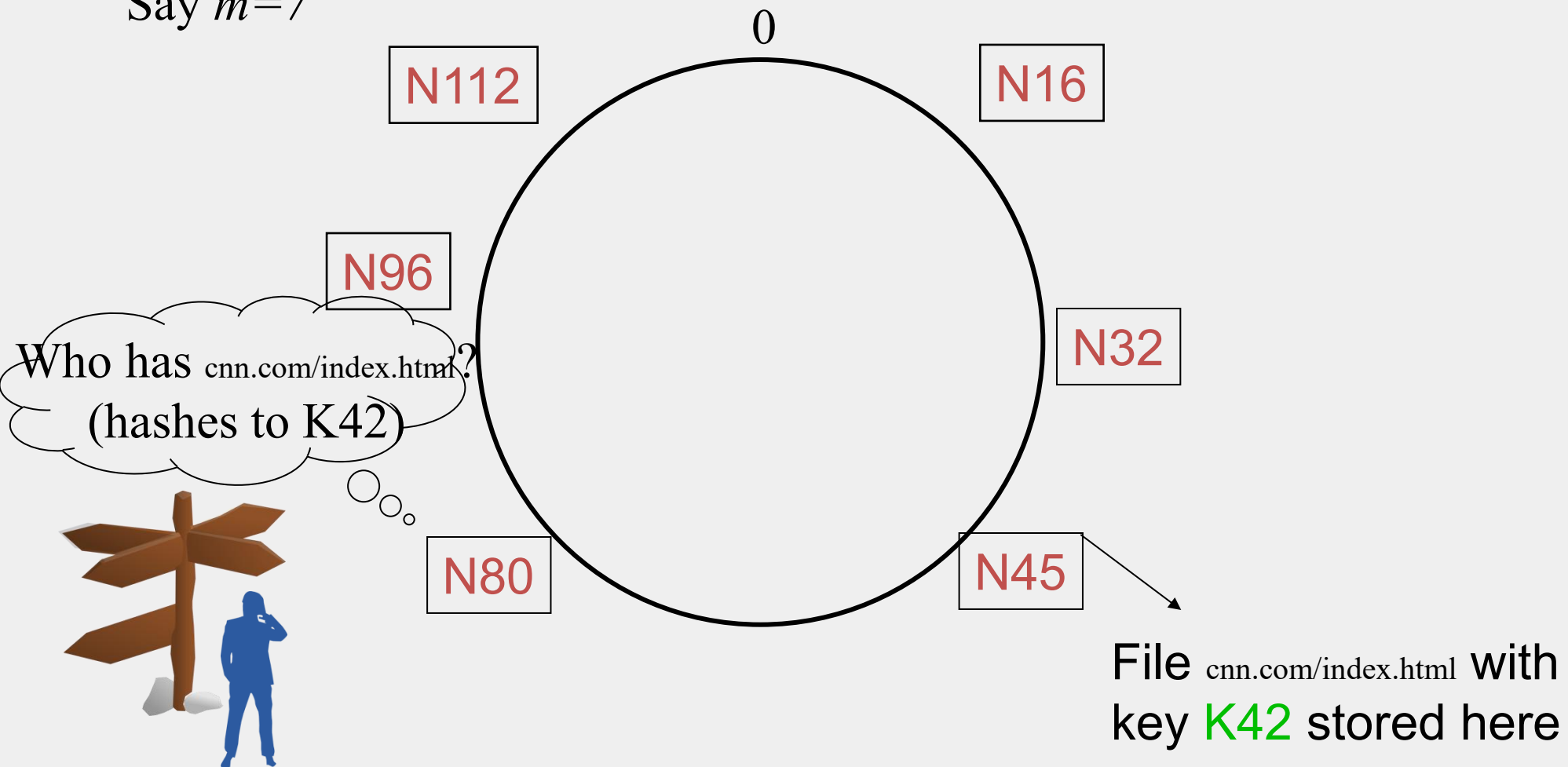
What about the files?

File is stored at first peer with id at or to the clockwise of $K/2$

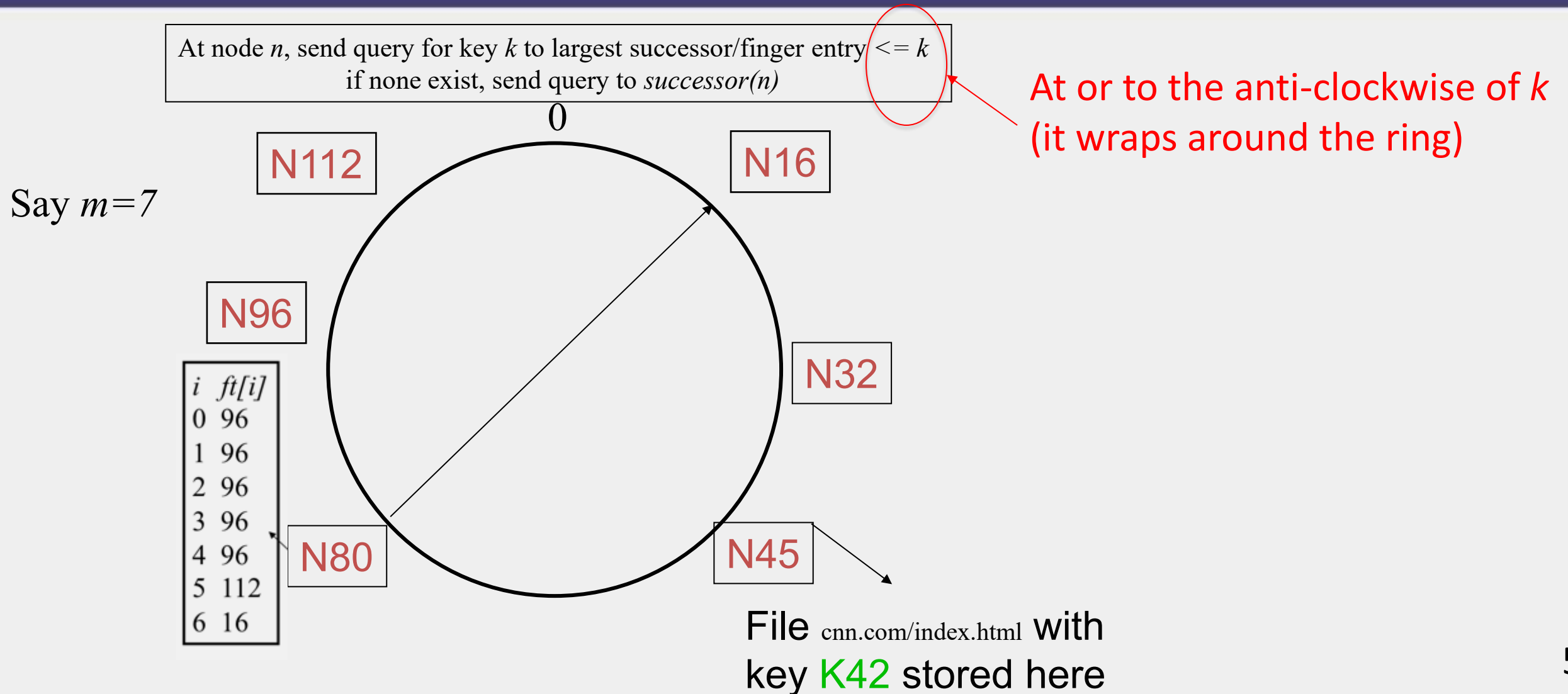
- *cooperative web caching*
- Consistent Hashing $\Rightarrow$ with K keys and N peers, each peer stores $O(K/N)$ keys. (i.e., $< c.K/N$, for some constant c)

Search

Say $m=7$



Search

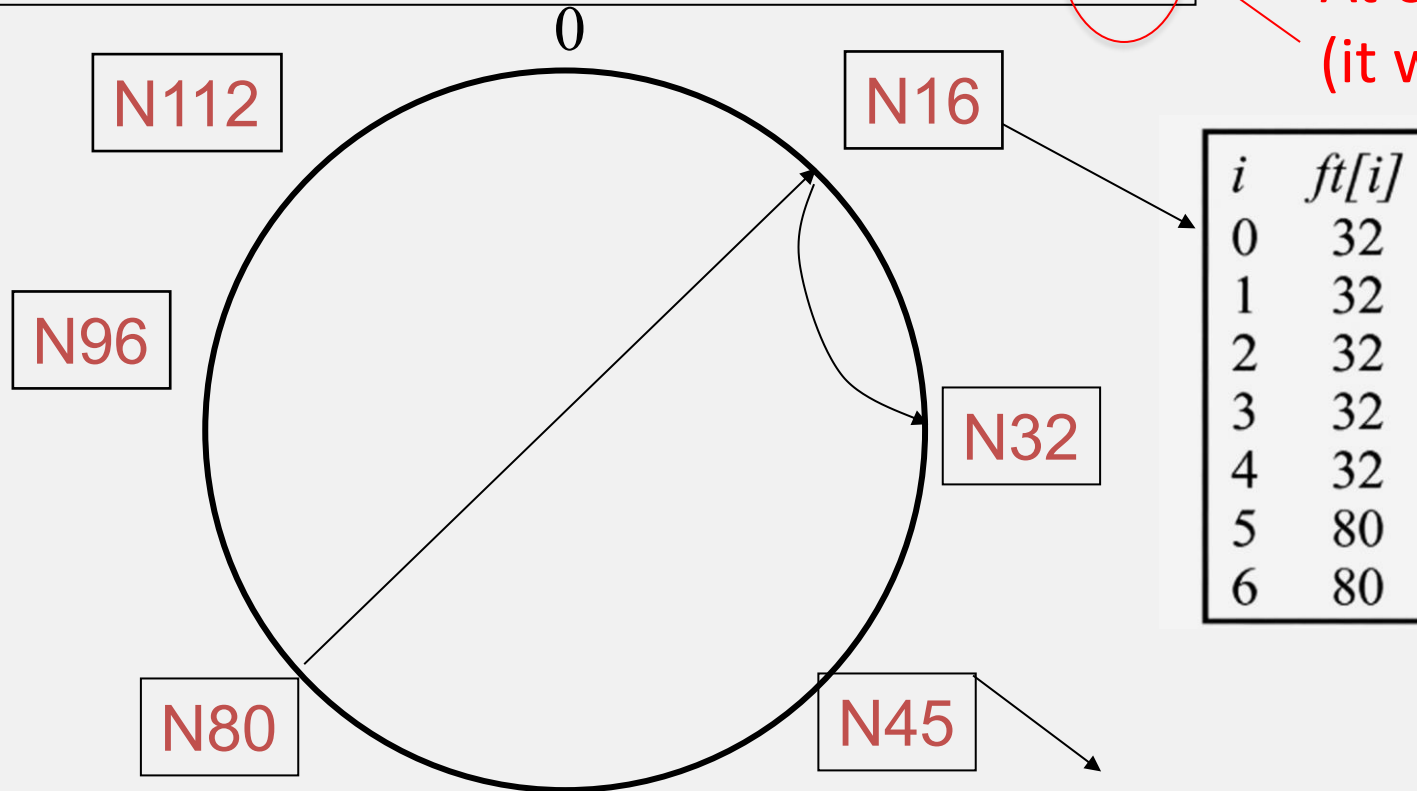


Search

At node n , send query for key k to largest successor/finger entry $\leq k$
if none exist, send query to $successor(n)$

At or to the anti-clockwise of k
(it wraps around the ring)

Say $m=7$



File cnn.com/index.html with
key **K42** stored here

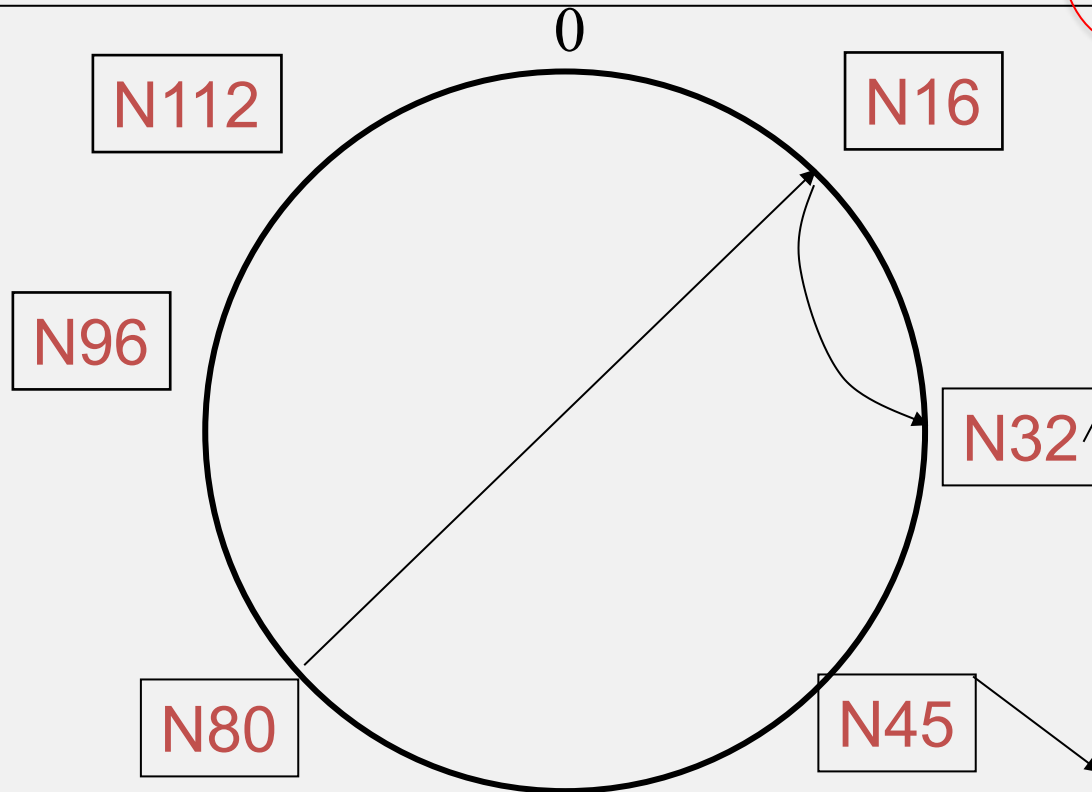


Search

At node n , send query for key k to largest successor/finger entry $\leq k$
if none exist, send query to $successor(n)$

At or to the anti-clockwise of k
(it wraps around the ring)

Say $m=7$



i	$ft[i]$
0	45
1	45
2	45
3	45
4	80
5	80
6	96

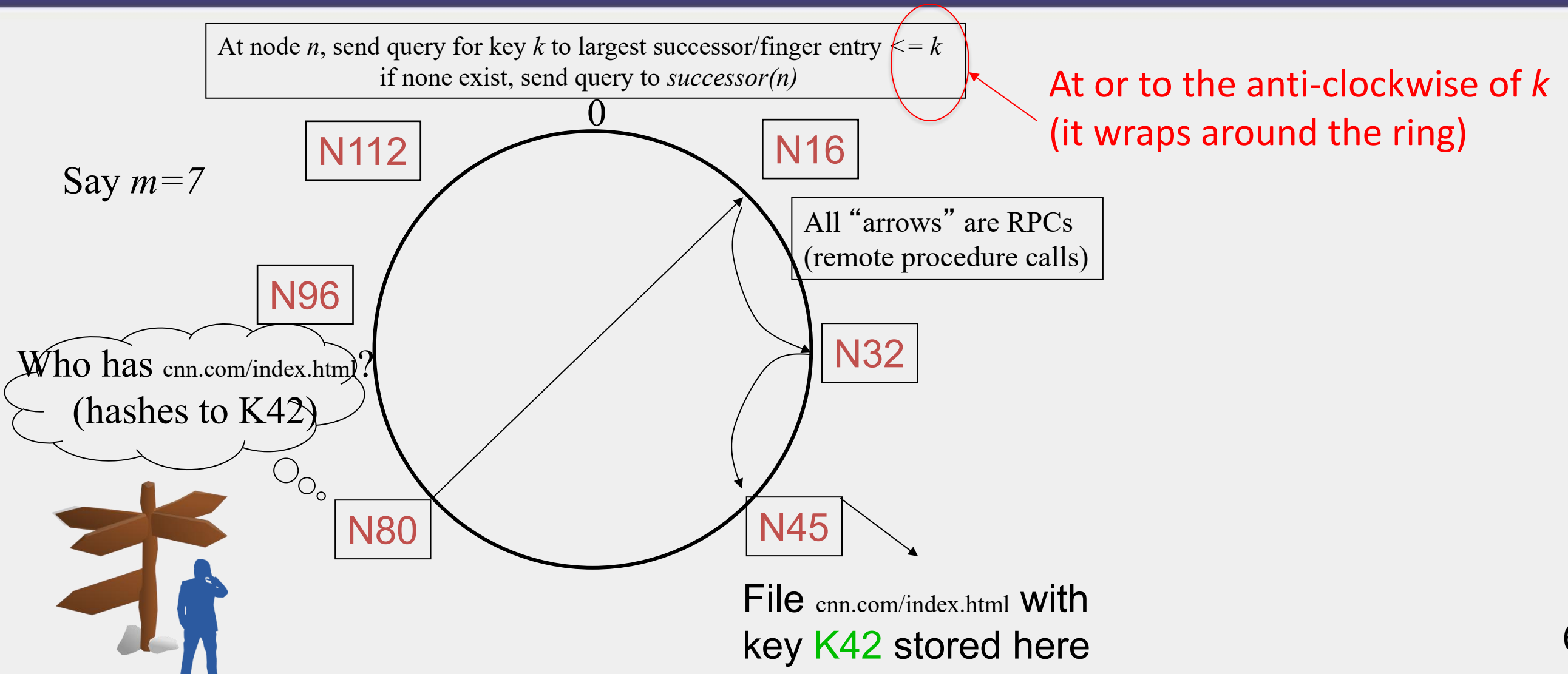
All finger table entries
overshoots query key k_{42}

So, route to successor

File cnn.com/index.html with
key **K42** stored here



Search



CS 425 / ECE 428

Distributed Systems

Fall 2026

Aishwarya Ganesan w/ Ram Kesavan

Lecture 7-8: Peer-to-peer Systems

Ack: Indranil Gupta

What We Are Studying

- Widely-deployed P2P Systems
 1. Napster
 2. Gnutella
 3. BitTorrent
- P2P Systems with Provable Properties
 1. Chord
 2. Kelips

Comparative Performance

	Memory	Lookup Latency	#Messages for a lookup	
Napster	$O(1)$ $(O(N)@server)$	$O(1)$	$O(1)$	
Gnutella	$O(N)$	$O(N)$	$O(N)$	
Chord	$O(\log(N))$	$O(\log(N))$	$O(\log(N))$	

Chord Recap...

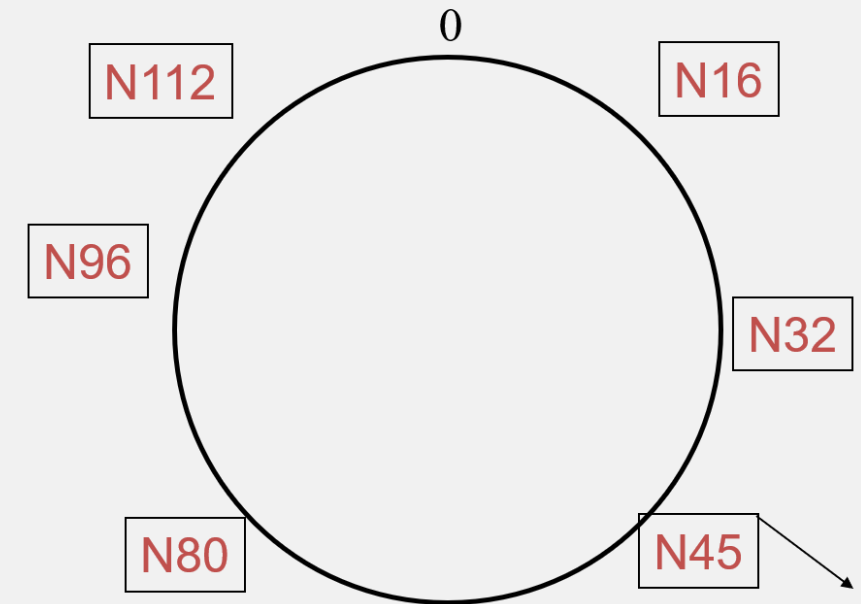
Consistent hashing

Map peer $\rightarrow$ m -bit nodeid

maps to one of 2^m logical points in a ring

File is stored at **successor of key mod 2^m**

Say $m=7$

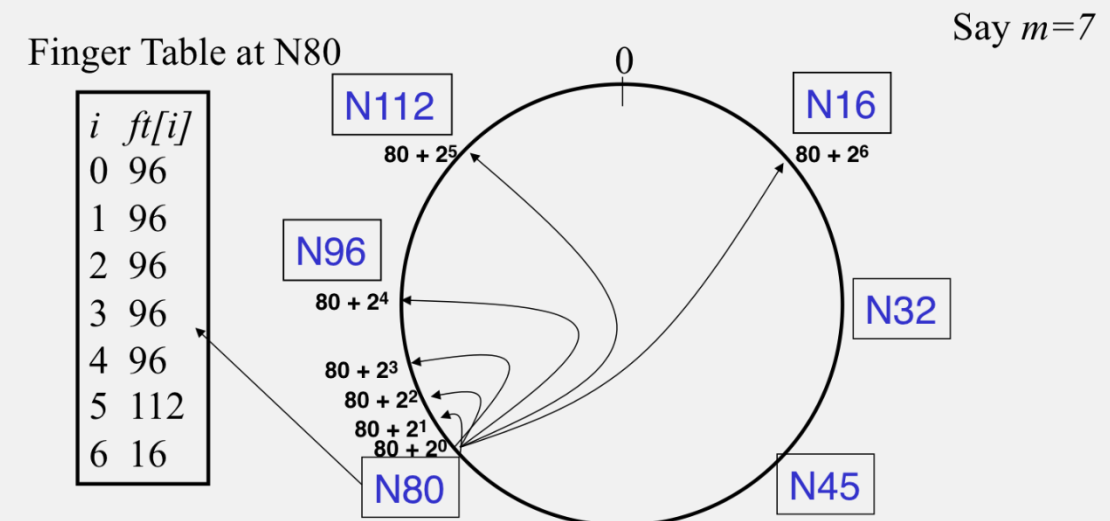


File cnn.com/index.html with
key **K42** stored here

Chord Recap...

Two kinds of peer pointers or neighbor info per peer

1. Successor
2. Finger-table entries



i^{th} entry at peer with id n is the first peer with id $\geq (n + 2^i) \bmod 2^m$

At or to the clockwise of

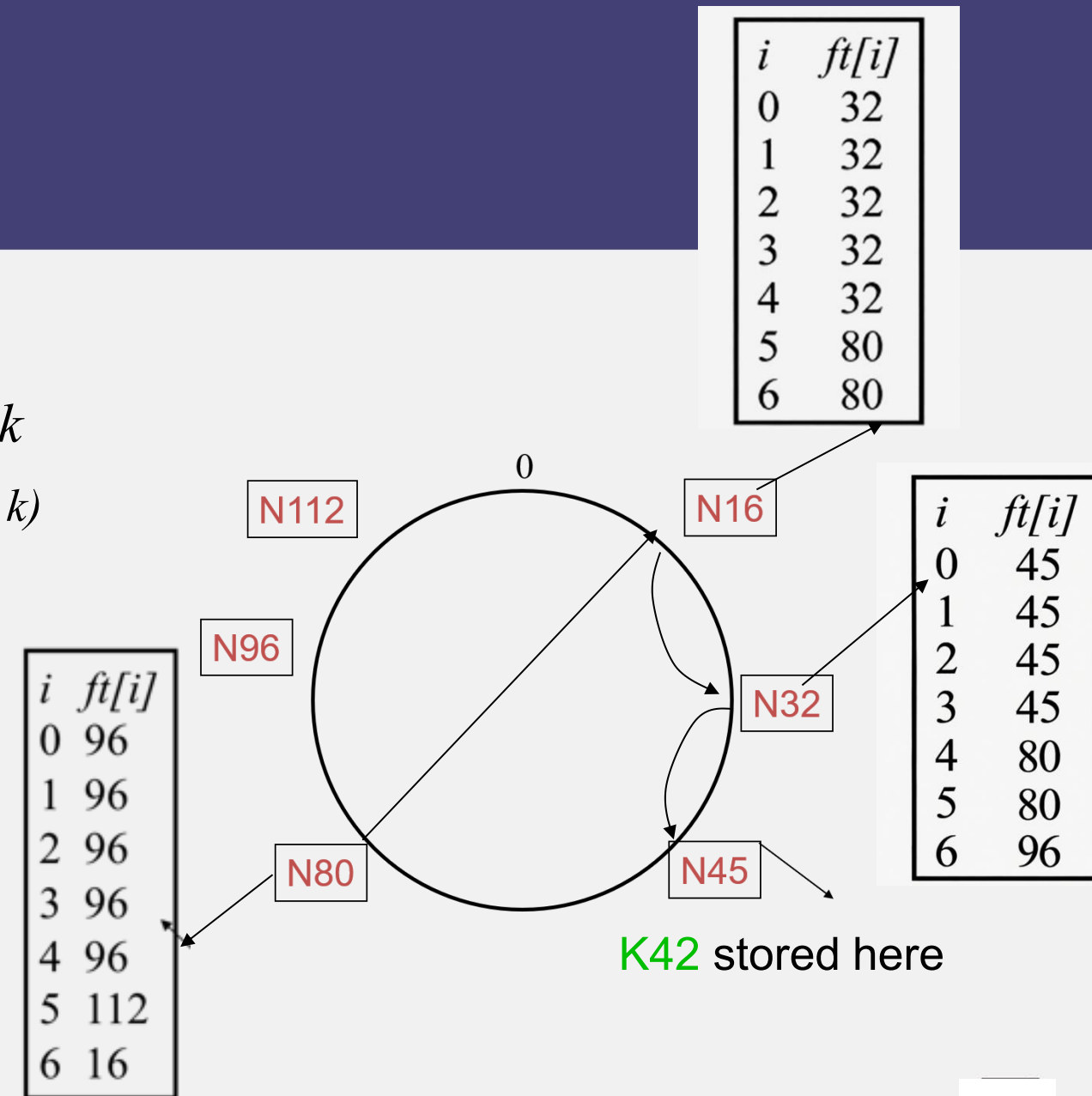
<https://pe.app/aishwaryaganesa>



Chord Recap...

Search for key k :

- At node n , send query to largest finger entry $\leq k$
i.e., largest finger entry between n and k (don't overshoot k)
Query(K42) at N80: N80 forwards to N16
Query(K42) at N16: N16 forwards to N32
- if none exist, send query to $successor(n)$
Query(K42) at N32: N32 forwards to successor (N45)



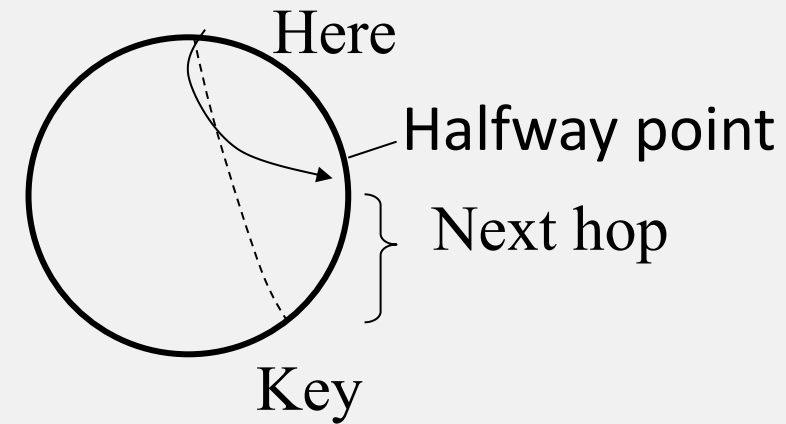
Analysis

Search takes $O(\log(N))$ time

Proof

- (intuition): *at each step, distance between query and peer-with-file reduces by a factor of at least 2*
- (intuition): after $\log(N)$ forwardings, distance to key is at most $2^m / 2^{\log(N)} = 2^m / N$

We will prove that the next hop of the query to key from node (here) is in the second half.



Analysis (contd.)

Search takes $O(\log(N))$ time

Proof

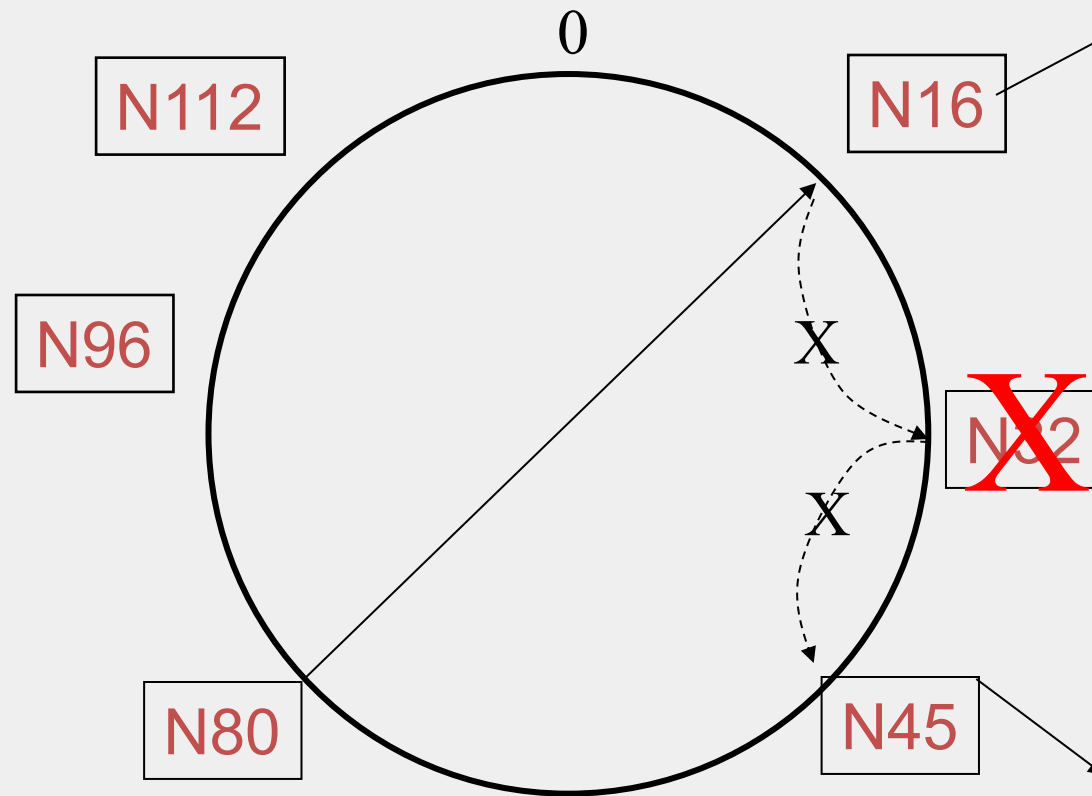
- After $\log(N)$ forwardings, distance to key is at most $2^m / 2^{\log(N)} = 2^m / N$
- Number of node identifiers in that range is $O(\log(N))$ with high probability
 - (why? SHA-1! and “Balls and Bins”)
- So, using *successors* in that range will be ok, using another $O(\log(N))$ hops

Analysis (contd.)

- $O(\log(N))$ search time holds for file insertions too
- In general, for *routing to any key*
 - “Routing” can thus be used as a **building block** for all operations: insert, lookup, delete
- $O(\log(N))$ time true only if finger and successor entries correct
 - When might these entries be wrong?
 - When you have failures

Search under peer failures - I

Say $m=7$



i	$ft[i]$
0	32
1	32
2	32
3	32
4	32
5	80
6	80

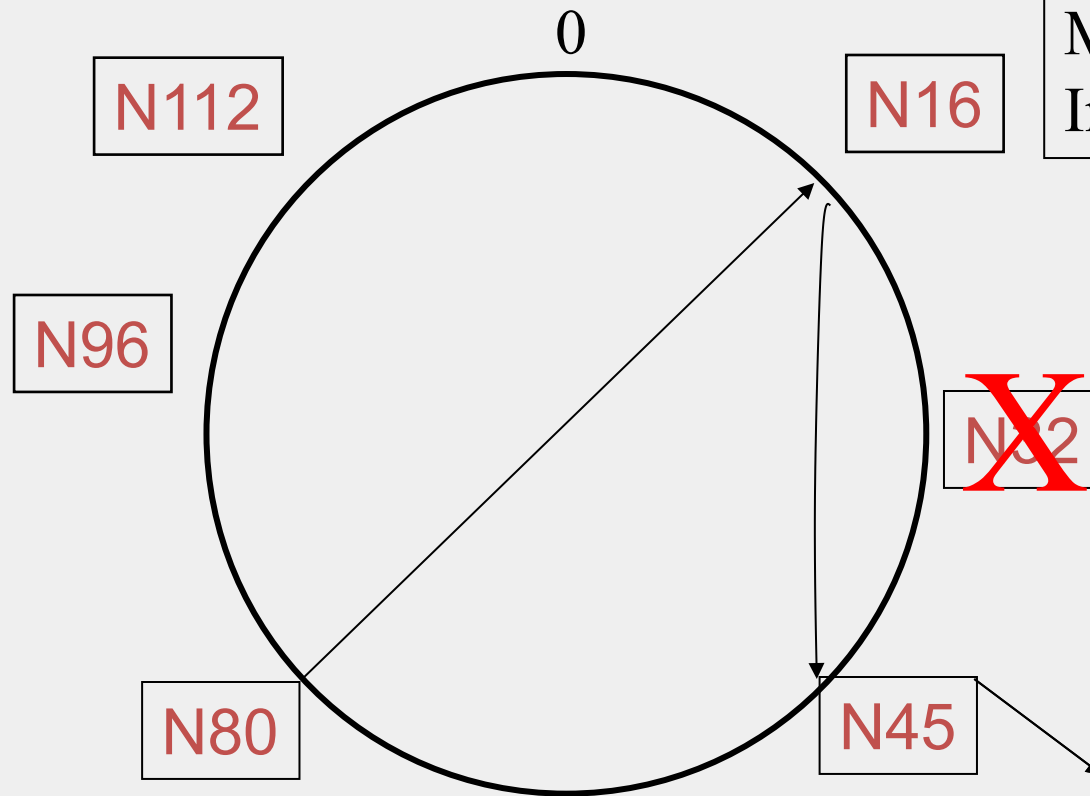
Lookup fails
(N16 does not know N45)

Search for K42

key **K42** stored here

Search under peer failures – I (solution)

Say $m=7$



One solution:

Maintain *r multiple* successor entries

In case of failure, use successor entries

Search for K42

key K42 stored here

Search under peer failures – I (solution)

- Choosing $r=2\log(N)$ suffices to maintain *lookup correctness* w.h.p.(i.e., ring connected)
 - Say 50% of nodes fail
 - $\Pr(\text{at given node, at least one successor alive})=$

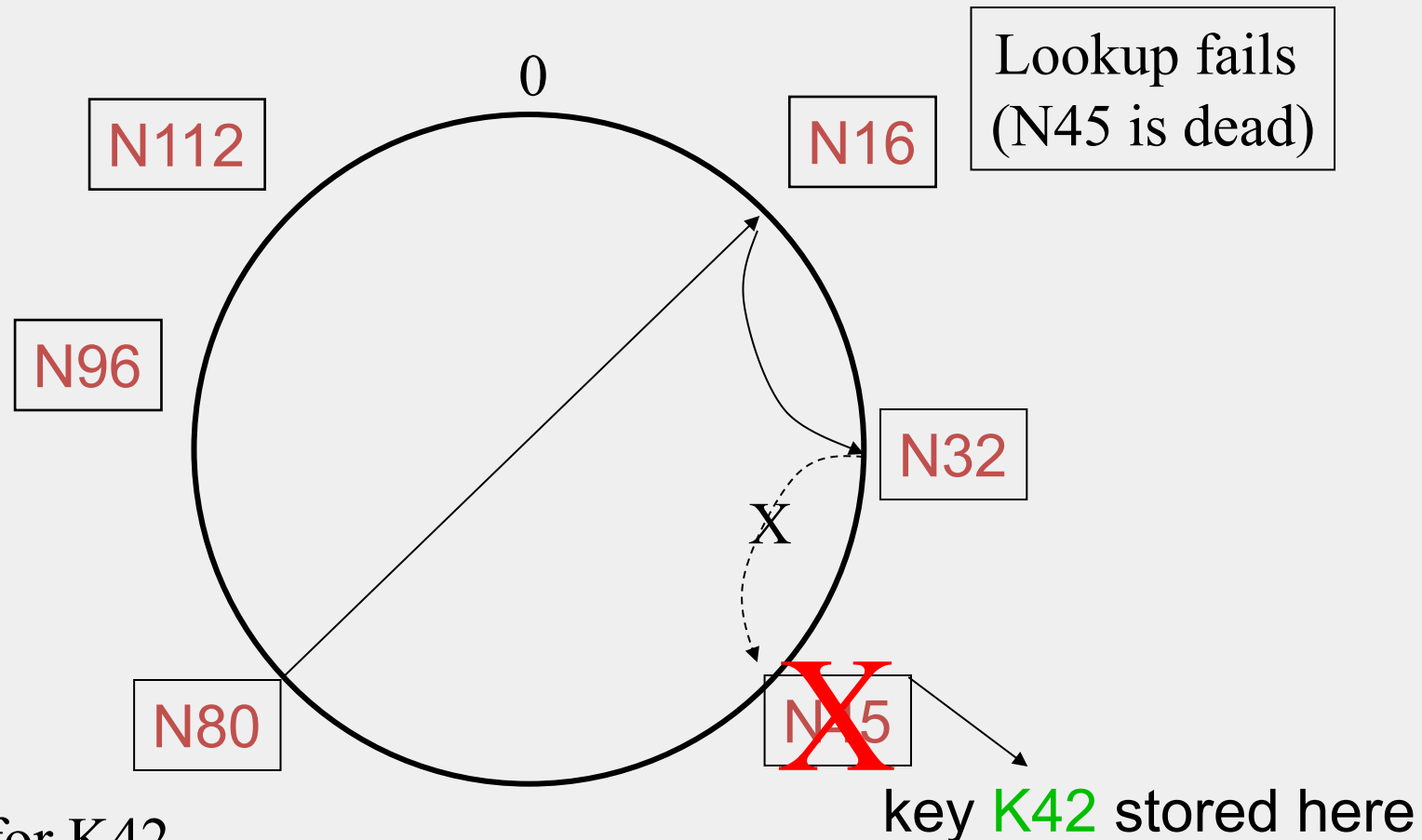
$$1 - \left(\frac{1}{2}\right)^{2\log N} = 1 - \frac{1}{N^2}$$

- $\Pr(\text{above is true at all alive nodes})=$

$$\left(1 - \frac{1}{N^2}\right)^{N/2} = e^{-\frac{1}{2N}} \approx 1$$

Search under peer failures - 2

Say $m=7$

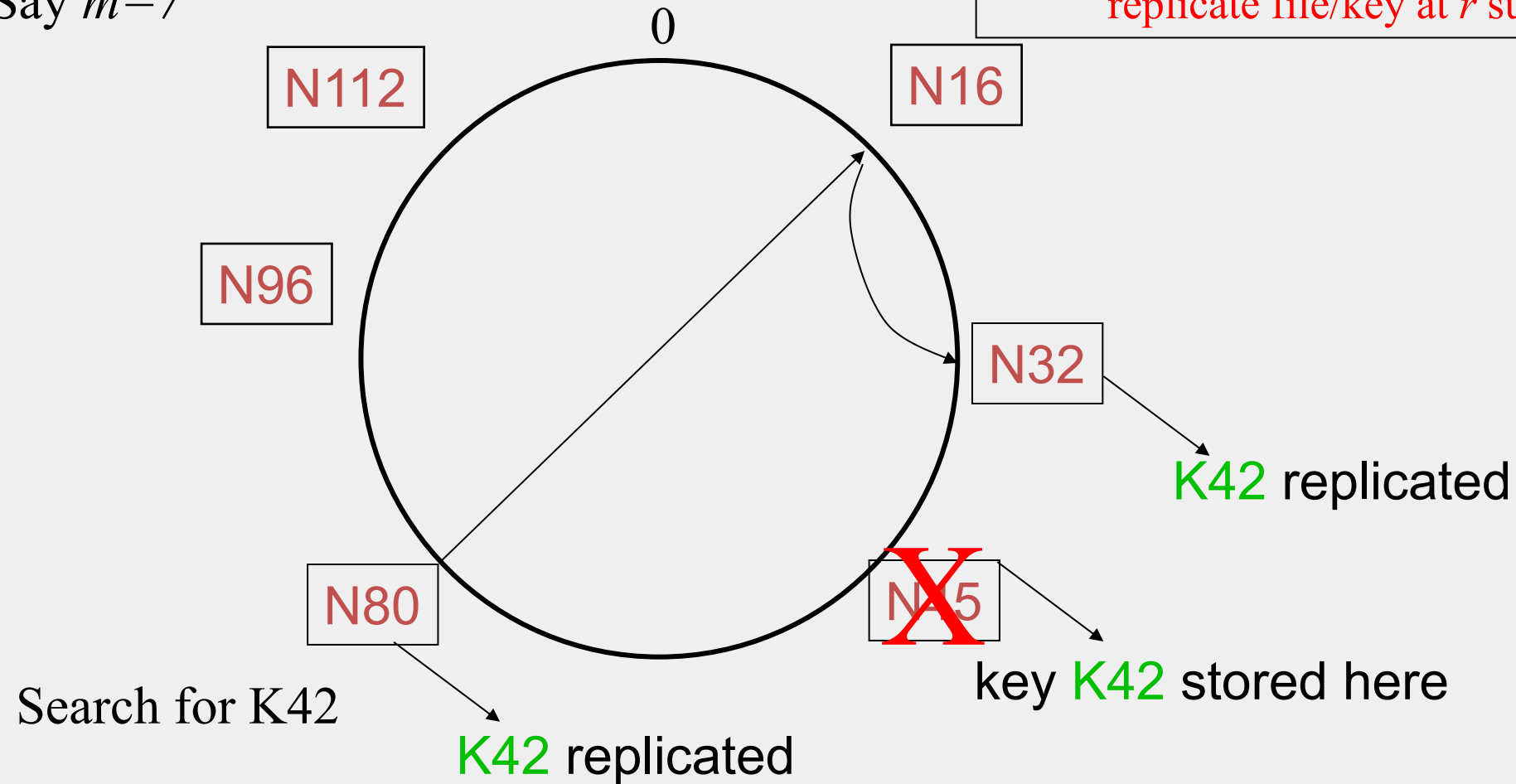


Search under peer failures -2: solution

Say $m=7$

One solution:

replicate file/key at r successors and predecessors



Need to deal with dynamic changes

? How did Gnutella deal with churn?

- ✓ Peers fail
 - New peers join
 - Peers leave
 - P2P systems have a high rate of *churn* (node join, leave and failure)
 - 25% per hour in Overnet (eDonkey)
 - 100% per hour in Gnutella
 - Lower in managed clusters

So, all the time, need to:

→ Need to update *successors* and *fingers*, and copy keys

Stabilization Protocol

Chord peers periodically run a *stabilization* algorithm that checks and updates pointers and keys

- Ensures *non-loopiness* of fingers, eventual success of lookups and $O(\log(N))$ lookups w.h.p.

Periodically, ask your **successor** for its predecessor.

- If that node lies between you and your successor, make it your new successor

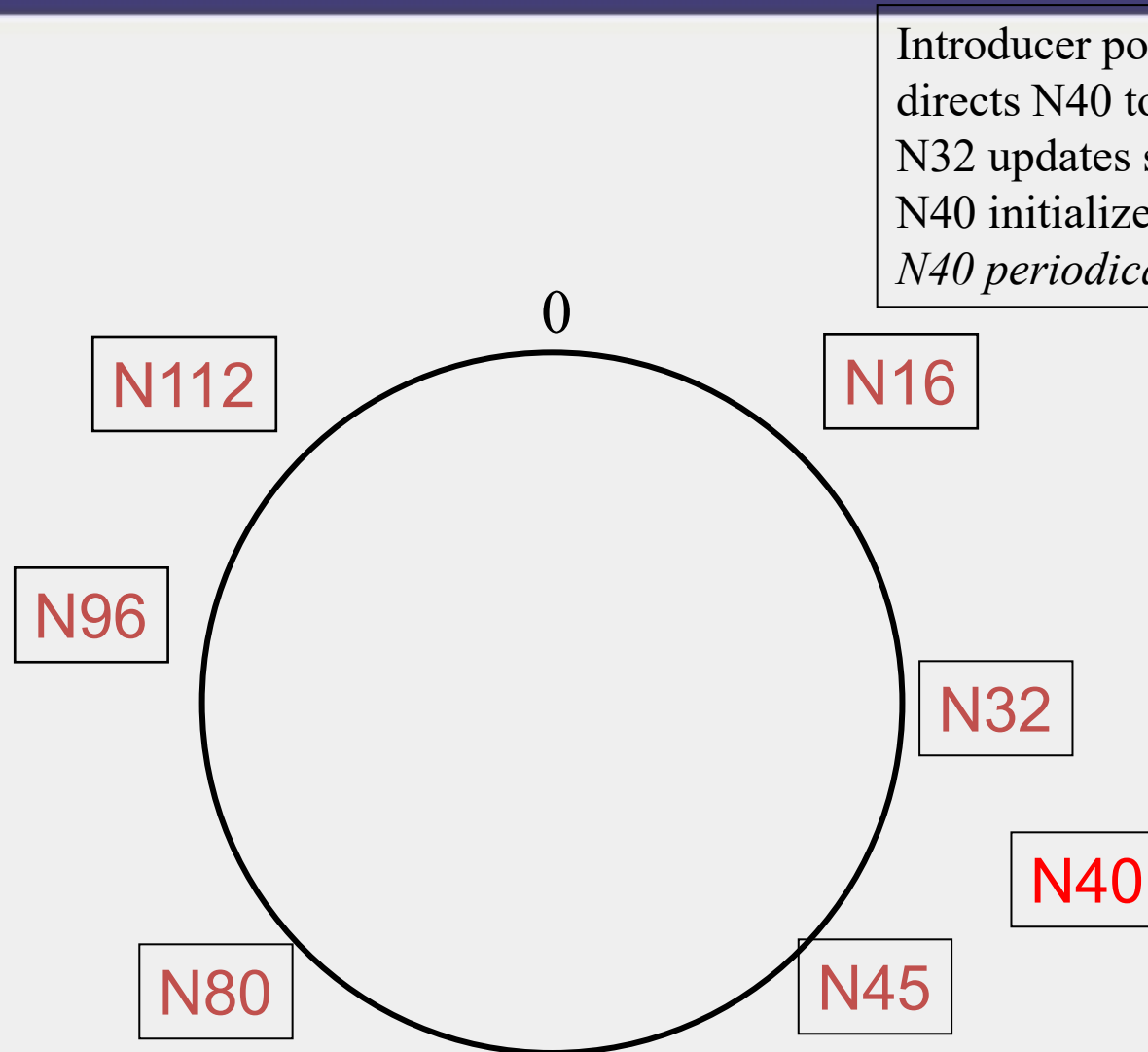
How to update fingertable entries?

- Route a query to $(n+2^i) \bmod 2^m$

New peers joining

? What should a peer (say N96) do when a new node N40 contacts it to join the system?

Say $m=7$



Introducer points to random peer
directs N40 to N45 (and N32)
N32 updates successor to N40
N40 initializes successor to N45, and inits fingers from it
N40 periodically talks to neighbors to update finger table

*Stabilization Protocol
(followed by all nodes)*

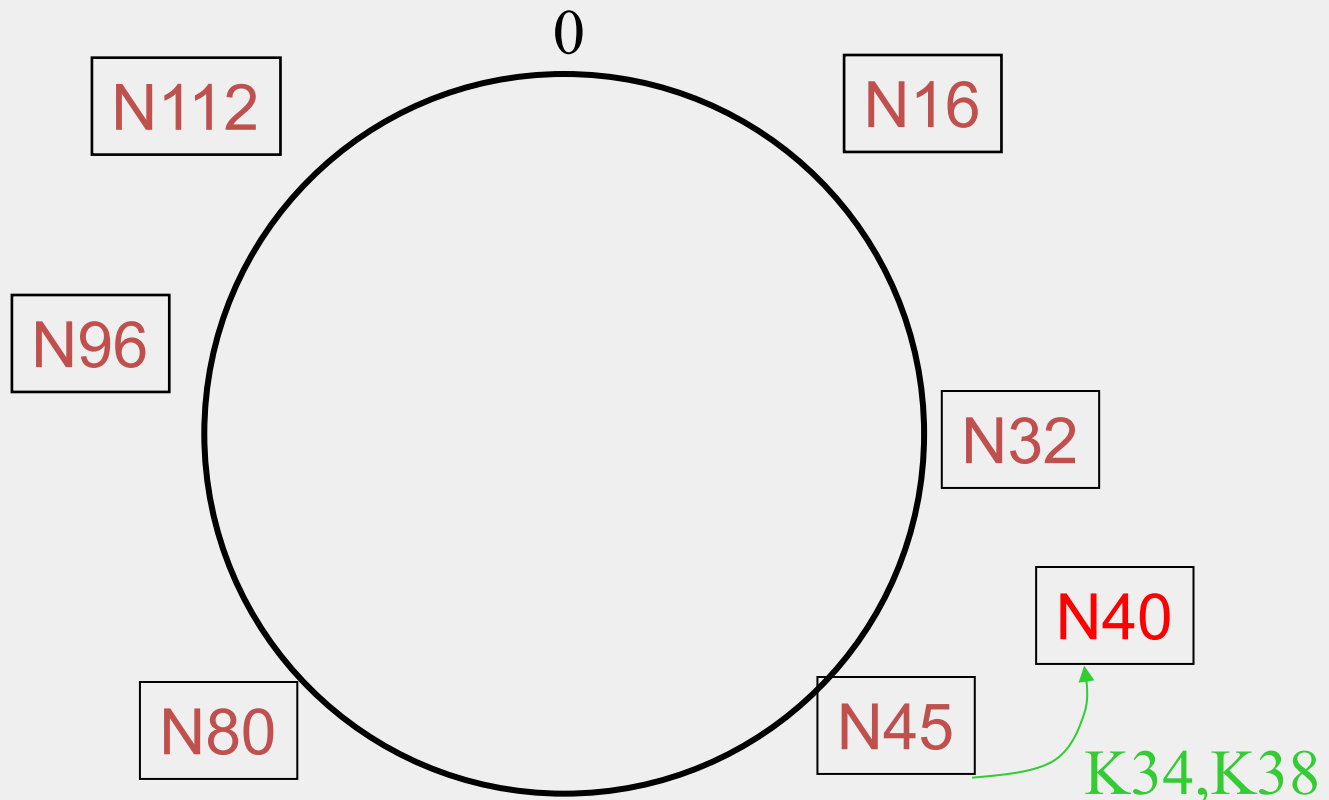
<https://pe.app/aishwaryaganesa>



New peers joining (2)

N40 may need to copy some files/keys from N45
(files with fileid between 32 and 40)

Say $m=7$



Advantage of consistent hashing

New peers joining (3)

- A new peer affects $O(\log(N))$ other finger entries in the system, on average [Why?]
- Number of messages per peer join = $O(\log(N) * \log(N))$
- Similar set of operations for dealing with peers leaving
 - For dealing with failures, also need *failure detectors* (you've seen them!)

Virtual Nodes

- Hash can get non-uniform → Bad load balancing
 - Treat each node as multiple virtual nodes behaving independently (typically $O(\log N)$)
 - Each virtual node maps to a point in the ring
 - Reduces variance of load imbalance

Churn

- When nodes are constantly joining, leaving, failing
 - Leads to excessive (unnecessary) key copying
 - Stabilization algorithm may need to consume more bandwidth to keep up
 - Strong stability takes $O(N^2)$ stabilization rounds
 - For more see [Extended paper on Chord webpage]
- Main issue is that files are replicated, while it might be sufficient to replicate only meta information about files
- Alternatives
 - Introduce a level of indirection, i.e., store only pointers to files (any p2p system)
 - Replicate metadata more, e.g., Kelips (later in this lecture)

Summary of Chord

$O(\log(N))$ lookup hops may be high:

- Topology-aware neighbor selection
 - Choose $\log N$ neighbors intelligently
 - You could modify Chord to do this
- Can we reduce the number of hops? – Kelips.

Chord Wrap-up Notes

More structured than Gnutella

Black box lookup algorithms

Churn handling can get complex

$O(\log(N))$ memory and lookup cost

Virtual Ring and Consistent Hashing used in Cassandra, Riak, Voldemort, DynamoDB, and other key-value stores

Current status of Chord project:

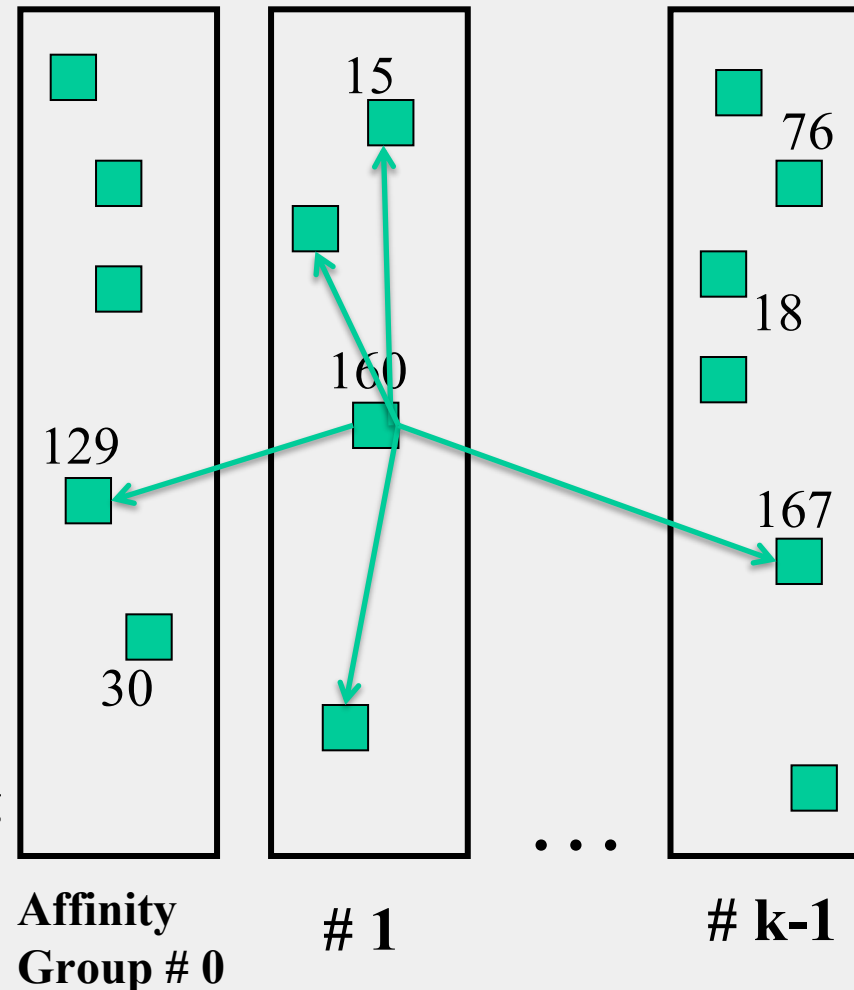
- File systems (CFS, Ivy) built on top of Chord
- DNS lookup service built on top of Chord
- Internet Indirection Infrastructure (I3) project at UCB
- Spawned research on many interesting issues about p2p systems

<https://github.com/sit/dht/wiki>

(Old: <http://www.pdos.lcs.mit.edu/chord/>)

Kelips – A 1 hop Lookup DHT

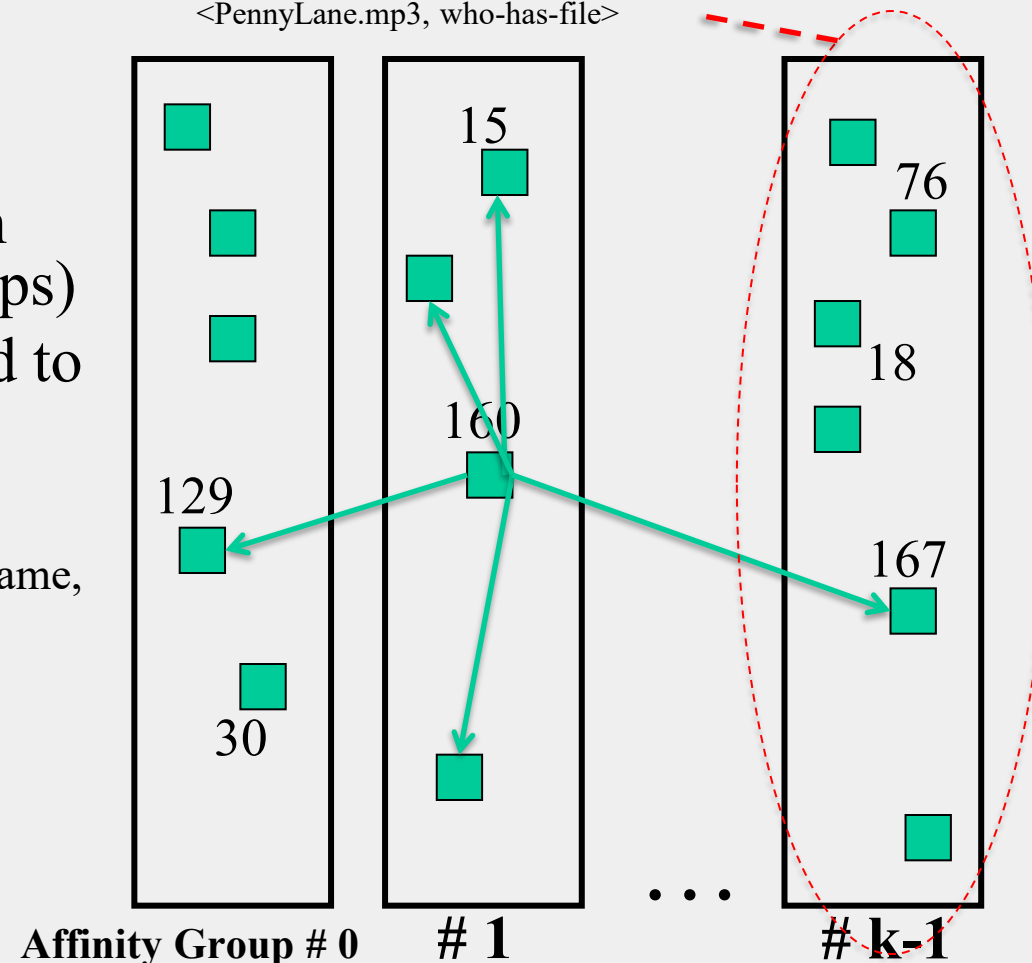
- k “affinity groups”
 - $k \sim \sqrt{N}$
- Each node hashed to a group (hash mod k)
- Node's neighbors
 - (Almost) all other nodes in its own affinity group
 - One contact node per foreign affinity group
 - Gossip-style heartbeating



Kelips Files and Metadata

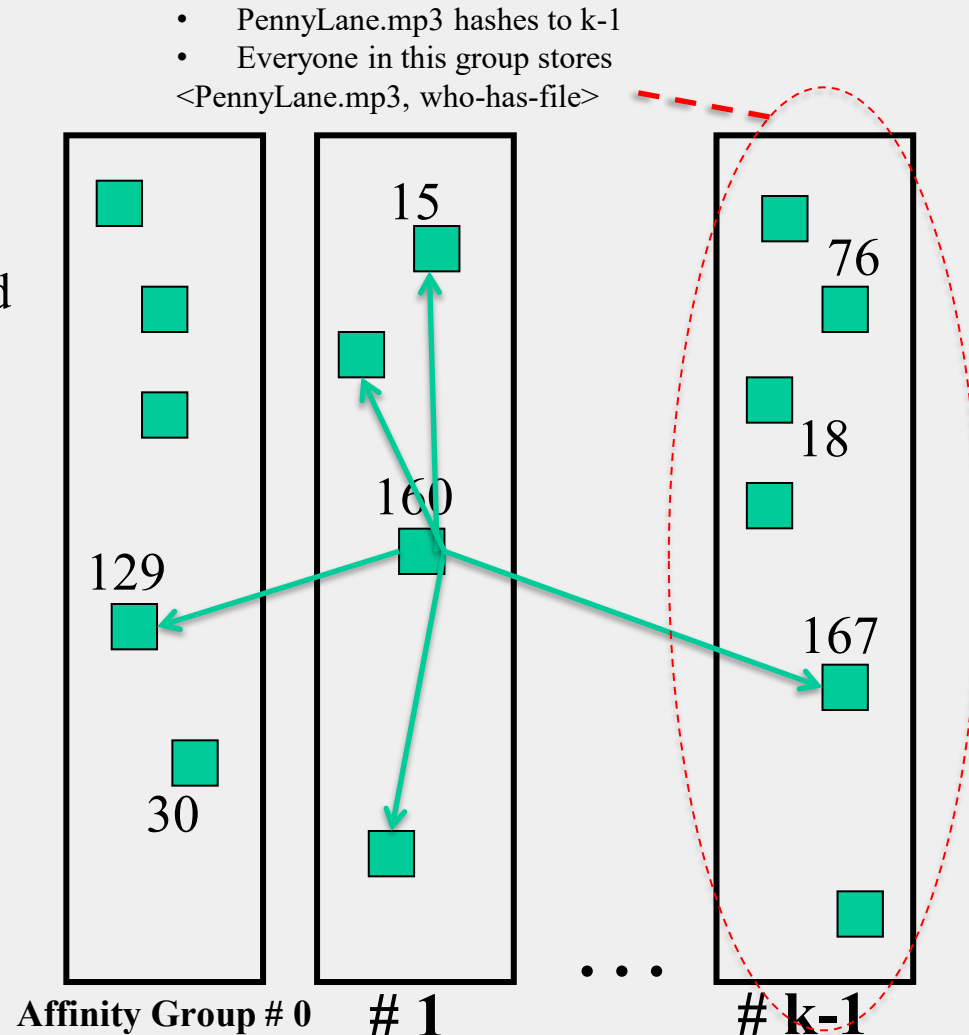
- File can be stored at any (few) node(s)
- **Decouple file replication/location** (outside Kelips) from file querying (in Kelips)
- Each filename hashed to a group
 - All nodes in the group replicate pointer information, i.e., <filename, file location>
 - Spread using gossip
 - Affinity group does not store files

- PennyLane.mp3 hashes to $k-1$
- Everyone in this group stores <PennyLane.mp3, who-has-file>



Kelips Lookup

- Lookup
 - Find file affinity group
 - Go to your contact for the file affinity group
 - Failing that try another of your neighbors to find a contact
- Lookup = 1 hop (or a few)
 - Memory cost $O(\sqrt{N})$
- 1.93 MB for 100K nodes, 10M files
- Fits in RAM of most workstations/laptops today (COTS machines)



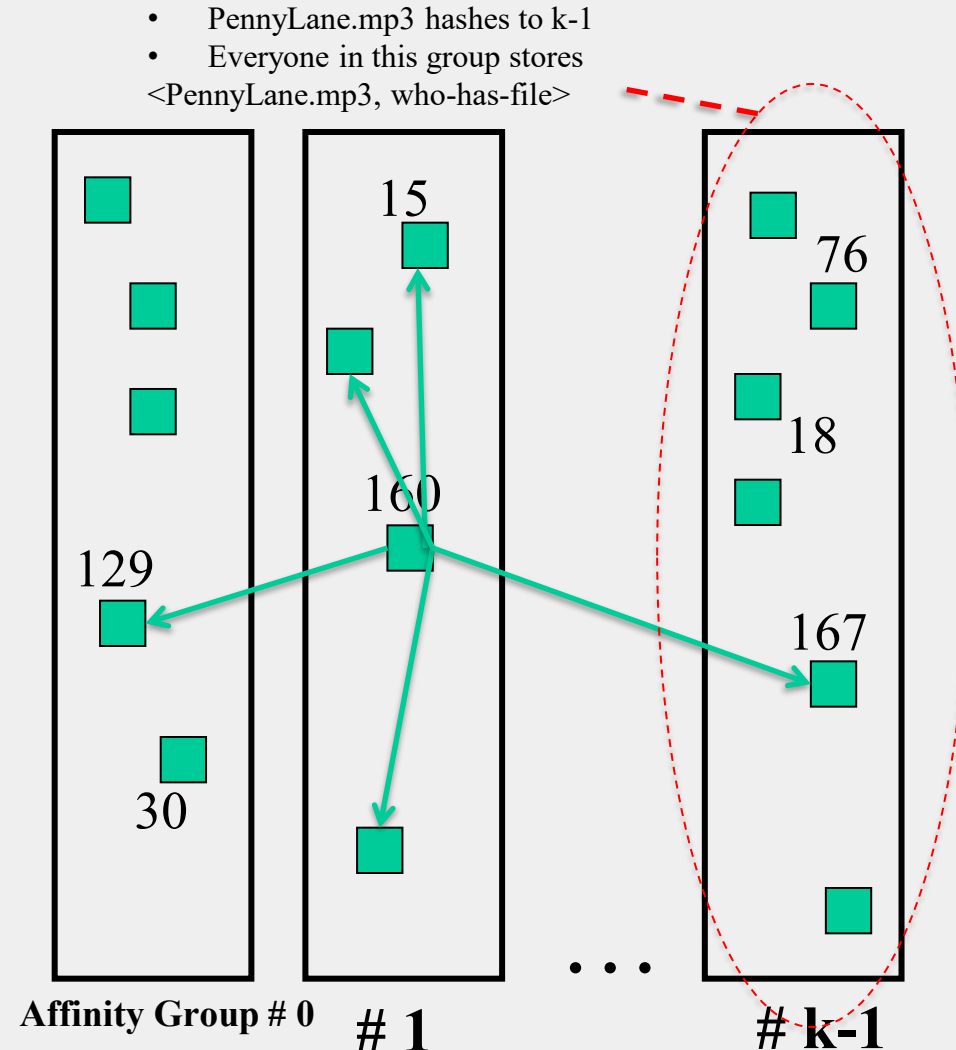
Kelips Soft State

- Membership lists

- Gossip-based membership
- Within each affinity group
- And also across affinity groups
- $O(\log(N))$ dissemination time

- File metadata

- Needs to be periodically refreshed from source node
- Times out



Chord vs. Kelips

- Range of tradeoffs available
 - Memory vs. lookup cost vs. background bandwidth (to keep neighbors fresh)

What We Have Studied

- Widely-deployed P2P Systems
 1. Napster
 2. Gnutella
 3. BitTorrent
- P2P Systems with Provable Properties
 1. Chord
 2. Kelips

Announcements

- MP2 out already, due 9/27 (demos on 9/28)
- HW2 due 10/4
- Both 11:59 pm on Sunday

Additional slides (not in syllabus)

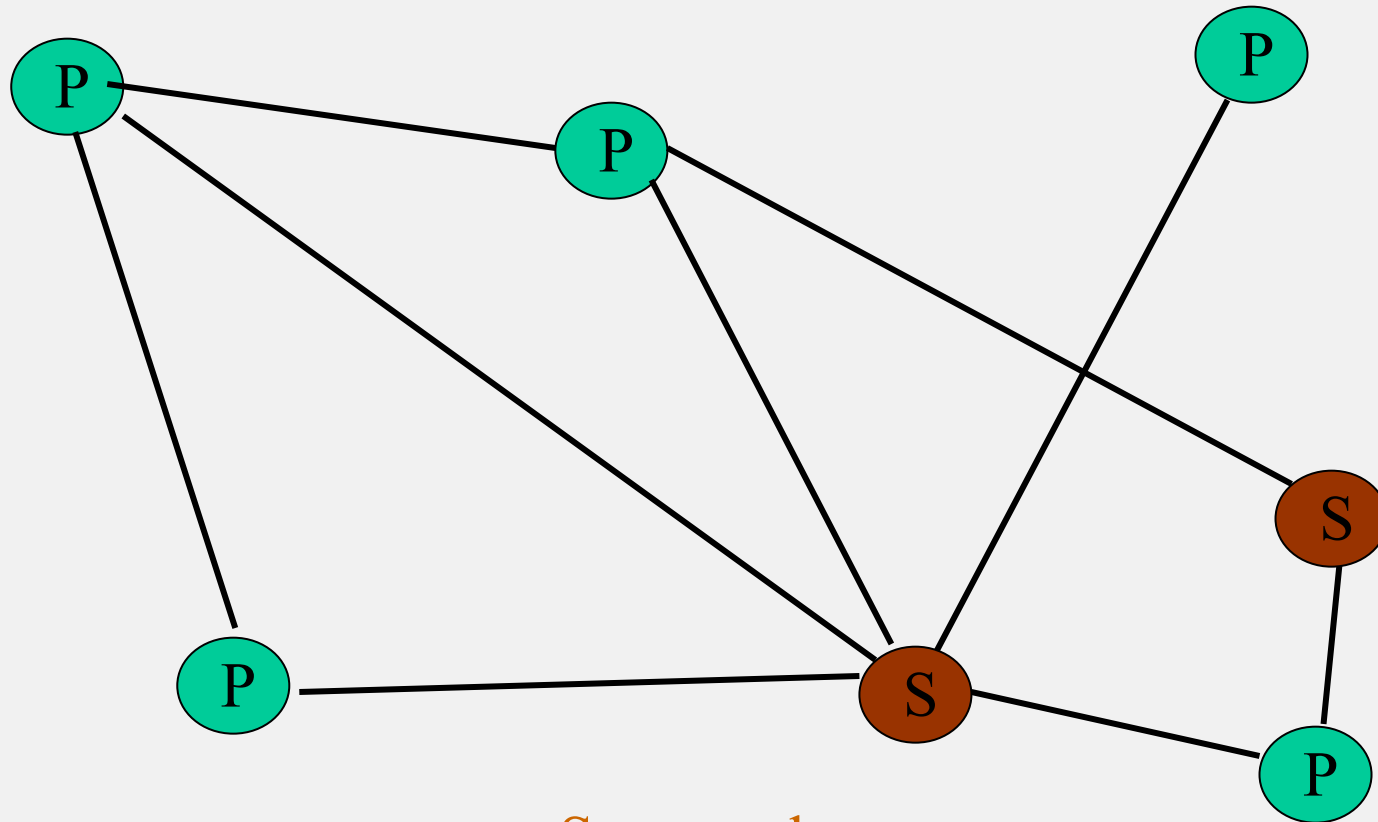
- Other widely-deployed P2P Systems
 1. Fasttrack
- Other P2P Systems with Provable Properties
 1. Pastry

FastTrack

- Hybrid between Gnutella and Napster
- Takes advantage of “healthier” participants in the system
- Underlying technology in Kazaa, KazaaLite, Grokster
- Proprietary protocol, but some details available
- Like Gnutella, but with some peers designated as *supernodes*

A FastTrack-like System

Peers



Supernodes

FastTrack (contd.)

- A supernode stores a directory listing a subset of nearby (<filename,peer pointer>), similar to Napster servers
- Supernode membership changes over time
- Any peer can become (and stay) a supernode, provided it has earned enough *reputation*
 - Kazaalite: participation level (=reputation) of a user between 0 and 1000, initially 10, then affected by length of periods of connectivity and total number of uploads
 - More sophisticated Reputation schemes invented, especially based on economics (See P2PEcon workshop)
- A peer searches by contacting a nearby supernode

Additional slides (not in syllabus)

- Other widely-deployed P2P Systems
 1. Fasttrack
- Other P2P Systems with Provable Properties
 1. Pastry

Pastry

- Designed by Anthony Rowstron (Microsoft Research) and Peter Druschel (Rice University)
- Assigns ids to nodes, just like Chord (using a virtual ring)
- **Leaf Set** - Each node knows its successor(s) and predecessor(s)

Pastry Neighbors

- **Routing tables** based on prefix matching
 - Think of a hypercube
- Routing is thus based on prefix matching, and is thus $\log(N)$
 - And hops are short (in the underlying network)

Pastry Neighbors

- Consider a peer with id 01110100101.
- Peer maintains a neighbor peer with an id matching each of the following prefixes (* = starting bit **differing** from this peer's corresponding bit):
 - *
 - 0*
 - 01*
 - 011*
 - ... 0111010010*

Pastry Routing

- Consider a peer with id 01110100101.
- Peer maintains a neighbor peer with an id matching each of the following prefixes (* = starting bit **differing** from this peer's corresponding bit):
 - *
 - 0*
 - 01*
 - 011*
 - ... 0111010010*
- When it needs to route to a peer, say 01110111001, it starts by forwarding to a neighbor with the **largest matching prefix**, i.e., 011101*
 - Hypercube routing

Pastry Locality

- For each prefix, say 011*, among all potential neighbors with the matching prefix, the neighbor with the **shortest round-trip-time** is selected
- Since shorter prefixes have many more candidates (spread out throughout the Internet), the neighbors for shorter prefixes are likely to be closer than the neighbors for longer prefixes
- Thus, in the prefix routing, **early hops are short and later hops are longer**
- Yet overall “stretch”, compared to direct Internet path, stays short